From attention to transformers lecture companion

One page per slide: a small capture of the slide, then the discussion the slide could not hold — what it shows, why it is true, the number it rests on, and where to read more.

PROF. TIMOTHY SMITH · MUMA COLLEGE OF BUSINESS, UNIVERSITY OF SOUTH FLORIDA
· COMPANION TO THE WEEK 5 DECK (40 SLIDES) · EVERY NUMBER HERE IS A STORED OUTPUT OF `LECTURE-NOTEBOOK.IPYNB`

How to use these notes

This is the companion to the Week 5 lecture. Each page shows one slide, small, and then discusses it at the length the slide cannot: what is on it, what it is claiming, the mechanism behind the claim, the number it rests on, and where the idea goes next. The slides are the skeleton; these pages are the flesh. Read them after class with the lecture notebook open, because every number quoted here was produced by a cell in that notebook, and re-running the cell is the fastest way to make a number your own.

Where the links go. Each page ends with a short *Go further* list. Those are optional. They point to the original papers, to the readings already on the course list, to documentation you will use in Assignment 5, and to a handful of explainers that do a particular idea better than a slide can. None of them is quiz material beyond what the lecture and the required readings already cover.

The argument of the week, in one paragraph

Week 4 ended with models that **read in order**: an RNN keeps one running memory; the LSTM and GRU protect it with gates so it fades far more slowly. Those gates did not change the design: everything the model has read still has to fit in one memory, the model can never look back at the input, and it works one word at a time. Block 1 states those three limits. Block 2 makes the first one visible on a small task (rewriting dates, 3.6% right) and fixes it with one idea, **attention**: let the decoder look back at every input position, score each one against what it is about to write, and blend. Same size, 97.6%. But attention did not fix the third limit: the model still works one step at a time, which a GPU cannot speed up. So Block 3 asks whether the RNN is needed at all, and builds the **transformer**: every word asks its own question of every other word, all at once, in one matrix multiplication. Position has to be added on purpose (without it, 34.5%). Block 4 shows that the same block, wired three ways, gives the whole family tree of today's language models: **encoders** that read both directions and return vectors, **decoders** that see only the past and write, and **encoder–decoders** that read everything, then write. Block 5 puts a decoder on your laptop.

Numbers to remember

What	Number	Where it comes from
Learned numbers in one recurrent layer (64 in, 64 memory)	RNN 8,320 · GRU 24,960 · LSTM 33,280	Week 4 notebook, §4
Week 4 ticket router, test accuracy	bag of words 0.8857 · plain RNN 0.7799 · LSTM 0.9693	Week 4 notebook, §5
Date rewriting, no attention, memory 16	**3.6%** exactly right, 12,317 learned numbers	§0.3
Same task, no attention, memory 64 / 128	80.3% / 99.1%, at 105,773 / 373,741 learned numbers	§0.3
Same task, memory 16, **with attention**	**97.6%**, still 12,317 learned numbers	§0.4
A small transformer, no RNN anywhere	**97.8%**, 63,053 learned numbers	§1.6

What	Number	Where it comes from
The same transformer with position switched off	**34.5%**	§2.1
Share of a transformer block's learned numbers in the feed-forward part	about two thirds (66.6%)	§1.4
Change in a decoder's earlier vectors when the last word changes	**exactly 0**	§3.2
Memory to hold an 8-billion-number model	32 GB at 4 bytes · 16 GB at 2 bytes · 4 GB at 4 bits	§4.4
24-task support benchmark	SmolLM2-135M 0.25 · Qwen2.5-0.5B 0.71 · Qwen2.5-1.5B 0.75	§6.3

Words used throughout

- **Vector** — a list of numbers. A word vector is a list of numbers standing for a word.
- **Learned numbers** — what engineers call *parameters* or *weights*: the numbers training adjusts. "0.5B" means half a billion of them.
- **Memory / hidden state** — the running summary an RNN carries from word to word.
- **Block / layer** — one repeatable unit of a transformer: attention, then a feed-forward network, with skip connections and normalisation around each.
- **Token** — the unit a model reads: a word, a piece of a word, or (in Blocks 2 and 3) a single character.

Contents

OPENING

1 From attention to transformers
2 One argument in three steps, then the family tree
3 One message, three jobs

BLOCK 1 · WHERE READING IN ORDER STOPS

4 Three readers, one idea: read in order, keep a memory
5 Three limits the gates did not remove

BLOCK 2 · ATTENTION

6 Two readers back to back: an encoder reads, a decoder writes
7 One summary is a bottleneck
8 Could we just buy a bigger memory?
9 The reader already made a memory after every word. We kept one.
10 Attention: weigh every memory by how much it matters to the question
11 Aside: the dot product measures agreement
12 The same three steps, by hand
13 Let the decoder look back at every input memory
14 Same size, same data, same training: one new connection
15 Nobody told it where to look
16 Two limits fixed. The RNNs still read in order.

BLOCK 3 · THE TRANSFORMER

17 One vector for “charge” — but three meanings
18 Every word asks a question; every word offers an answer
19 Every word asks at once: one table, one matrix multiplication
20 Attention alone is a bag of words. Order must be added on purpose.
21 One block: words pull in context, then each word consults what it knows
22 The 2017 diagram, box by box
23 The same machine as inputs and outputs

BLOCK 4 · ENCODERS AND DECODERS

24 Which shape for which job
25 Today’s small language models, by shape

BLOCK 5 · A LANGUAGE MODEL ON YOUR LAPTOP

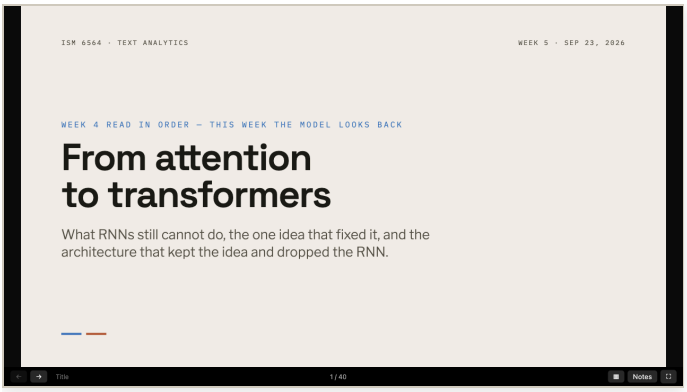
26 A decoder becomes an assistant in two stages
27 Decoding: how the next token gets picked
28 Running a model: two bills, memory and time
29 Small vs large is a measurement, not an opinion

WRAP-UP

30 Design Northwind’s support workflow
31 The climb so far
32 This week

APPENDIX · THE TABLE COMPUTED, TWO REFINEMENTS, REAL ATTENTION, AND THE CODE BEHIND THE SLIDES

33 The code behind the slides
34 Why decoder-only won — and where the encoders went
35 What changed, side by side
36 Is that only a story? Four lines compute the table — no loop
37 One question per word is only one kind of relevance
38 Real attention, in a real model
39 Attention in the date model: three lines
40 The date transformer



SLIDE 1 OF 40 · OPENING

From attention to transformers

The week in one line: the RNN ran out of road, attention fixed the part that mattered, and the transformer kept attention and dropped the RNN.

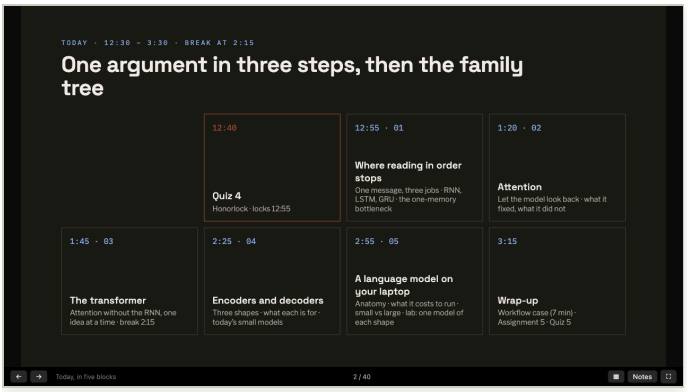
The title is a sentence, not a topic, and the sentence is the week's argument: Week 4 read in order; this week the model **looks back**. Everything that follows is a chain of small problems and small fixes. If you lose the thread at any point, come back to this page and find where you are in the chain.

Three things are worth doing before you read on. First, open the Week 5 lecture notebook beside these notes; the slides quote its outputs and the notebook has the code that made them. Second, have last week's three results in your head — plain RNN 0.78, LSTM 0.97 on the ticket router, and the plain RNN's memory fading to nothing within about ten words — because Block 1 starts from them. Third, decide now that you will not try to memorise the mechanism on the first pass. Every mechanism in this deck is introduced twice: once as a problem and a picture, once as a measurement. Understand the problem first; the mechanism then has somewhere to attach.

The subtitle names the three moves: *what RNNs still cannot do, the one idea that fixed it, and the architecture that kept the idea and dropped the RNN*. Those are Blocks 1, 2 and 3. Blocks 4 and 5 are about the models that architecture produced and how to run one.

GO FURTHER

- Week 4 lecture notebook, §3–§5 — the RNN, LSTM and router results this week starts from.
- Jurafsky & Martin, *Speech and Language Processing*, ch. 8 "Transformers" — the required chapter; read it after the lecture, not before.



SLIDE 2 OF 40 · OPENING

12:55 · 01

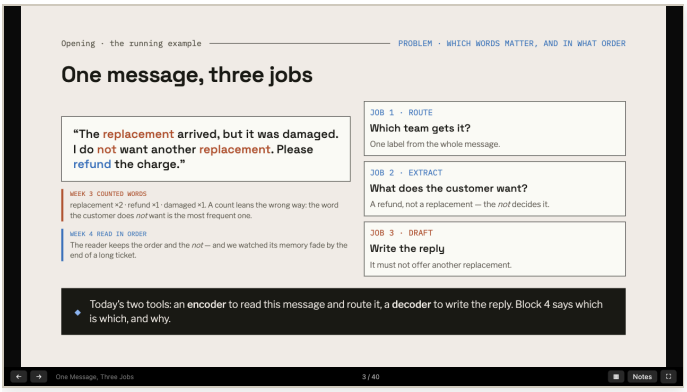
One argument in three steps, then the family tree

Five blocks; the first three are one continuous argument, the fourth is the family tree, the fifth is hands-on.

The agenda is arranged so that each block ends on the problem the next block solves. Block 1 ends with three limits that gates did not remove. Block 2 fixes two of them with attention and ends on the third. Block 3 fixes the third by removing the RNN and ends on a question — which half of the resulting machine do you need? — that Block 4 answers. Block 5 is the practical coda: what one of these models costs to run, and how to run it.

The clock matters for how to read the deck. Three minutes on the running example, then roughly twenty-five minutes each for Blocks 1 to 3, with the break falling just after the transformer is assembled; thirty minutes on Block 4, because the encoder/decoder distinction is the single thing from this week you will use every remaining week of the course; twenty on Block 5, which feeds Assignment 5 directly; and the last fifteen on the group case and the wrap-up.

Quiz 5, written next week, covers this lecture, the lecture notebook, and the required readings. It follows the shape of Quizzes 2 to 4: six multiple choice, two fill-in, one written question worth two points. The written question will ask you to explain a mechanism or to choose a model shape for a job and defend the choice — the two skills Blocks 3 and 4 practise.



SLIDE 3 OF 40 · OPENING

PROBLEM · WHICH WORDS MATTER, AND IN WHAT ORDER

One message, three jobs

One customer message, three jobs a support system does with it, and the reason counts lean the wrong way: the word the customer does not want is the most frequent one.

This is the running example for the afternoon, and every model in the deck is judged against it. *The replacement arrived, but it was damaged. I do not want another replacement. Please refund the charge.* Three things a support system has to do with that message. **Route** it: which team gets it, one label from the whole text. **Extract** what the customer wants: a refund, not a replacement. **Draft** the reply, which must not offer another replacement.

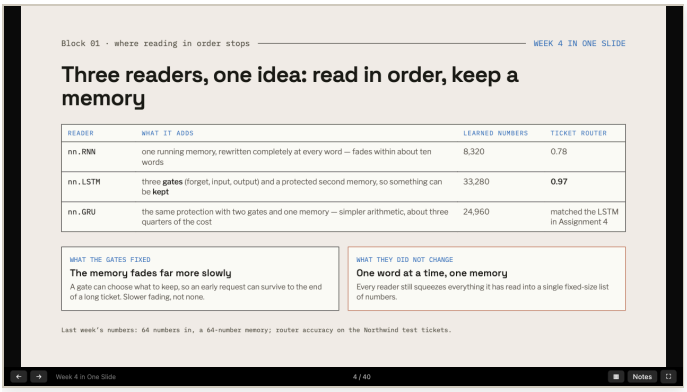
Now count the words, which is what Week 3 did. *Replacement* appears twice, *refund* once, *damaged* once. A bag of words has no way to see that the most frequent word is the one the customer is refusing: the *not* two words earlier is just another count, and order is gone. That is not a claim about any particular router's output; it is a fact about counts. Week 4's reader keeps order, so the *not* can do its work, but you also watched its memory fade by the end of a long ticket, and a real ticket is longer than this one.

Hold the message in mind through the blocks that follow, because each is a response to it. Block 2 gives a model a way to look back at the exact words that matter. Block 3 removes the reading-in-order that made looking back slow. Block 4 turns the result into two tools: an **encoder** that reads the whole message both ways and returns a vector to route it with, and a **decoder** that writes the reply one token at a time. Block 5 asks what running either on your own machine costs. The closing group case designs the whole workflow around this message and its relatives.

Two habits to start now. First, context is a capability, not a guarantee: a model that *can* use the *not* may still route this ticket to Returns, which is why every block ends with a measurement rather than a promise. Second, the three jobs are three different outputs, a label, a field and a paragraph, and the week's central lesson is that different shapes of model are built for different outputs.

GO FURTHER

- Week 3 assignment — the TF-IDF ticket router that saw only counts; the honest baseline for the closing case.
- Jurafsky & Martin, ch. 4 "Naive Bayes, Text Classification, and Sentiment", §4.4 — the classic patch for negation in a bag of words, and why it only goes so far.



SLIDE 4 OF 40 · BLOCK 01 · WHERE READING IN ORDER STOPS

WEEK 4 IN ONE SLIDE

Three readers, one idea: read in order, keep a memory

Three readers, one idea: read one word at a time and carry a memory forward. The gates slowed the fading; they did not change the design.

The table compresses Week 4 to one row per reader. All three share the idea in the title: the model reads the words in order and keeps a **running memory**, a short list of numbers (64 of them in our models) that summarises everything read so far. What differs is how that memory is updated.

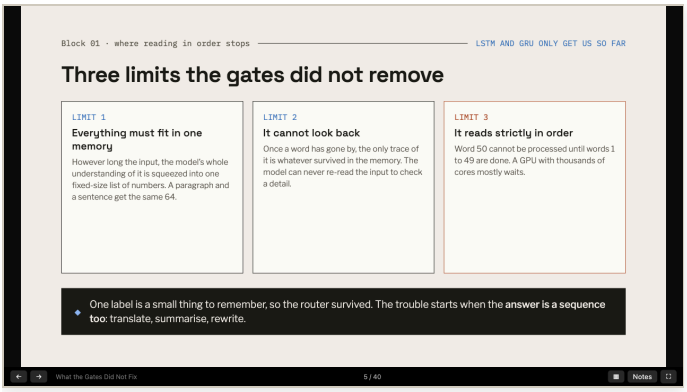
A **plain RNN** rewrites its whole memory at every word. That is why it forgets: each rewrite scales the trace of earlier words by a factor smaller than one, and Week 4 measured the first word's influence at about 10^{-18} of the last word's after forty words — zero for any practical purpose. The **LSTM** adds three learned switches, the *gates* (forget, input, output), and a second, protected memory that the gates control. Because the model can choose to leave that memory unchanged for many words, something read early can survive to the end. Slowed, not removed: the gates are learned and they leak, so over hundreds of words the early trace still thins out, and everything read is still squeezed into the same fixed-size memory. The **GRU** achieves nearly the same protection with two gates and one memory, which makes it about three quarters of the LSTM's size and a little faster to train.

The *learned numbers* column is what engineers call parameters: 8,320 for the RNN, 24,960 for the GRU, 33,280 for the LSTM, all with 64 inputs and a 64-number memory. The gates are where the extra numbers go. The last column is the payoff on our ticket router: the plain RNN scored 0.78, worse than the bag of words (0.89), because by the end of a long ticket the first request had faded; the LSTM scored 0.97, because it could hold the first request to the end. The GRU was not run on the router in the lecture; when you swapped it in for Assignment 4 it matched the LSTM.

Two things to carry forward. First, in PyTorch the three are interchangeable — `nn.RNN`, `nn.GRU`, `nn.LSTM`, one word of code — and everything else in the pipeline stays the same. Second, and this is the hinge of the whole week: the gates protect the memory, but it is still **one** memory, and the reading is still **in order**. The next slide draws out what that implies.

GO FURTHER

- Christopher Olah, "Understanding LSTM Networks" (2015) — still the clearest picture of the gates.
- Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder" (2014) — the paper that introduced the GRU, and also the encoder–decoder used in Block 2.



SLIDE 5 OF 40 · BLOCK 01 · WHERE READING IN ORDER STOPS

LSTM AND GRU ONLY GET US SO FAR

Three limits the gates did not remove

Gates stopped the memory fading. Three limits remain, and all three come from reading one word at a time through one memory.

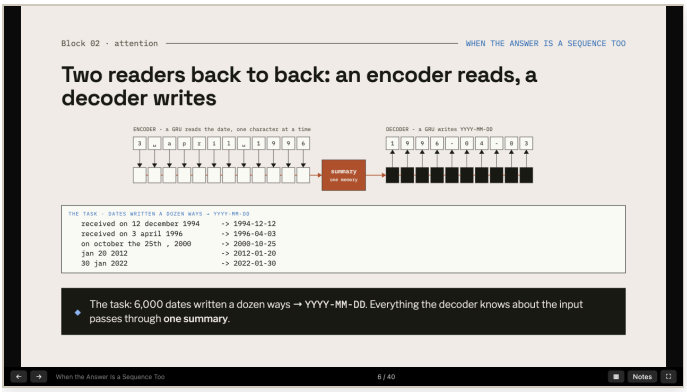
This is the problem statement for the whole week, and it is worth being precise about each limit, because Blocks 2 and 3 will each fix specific ones.

Limit 1 — everything must fit in one memory. The memory is a fixed-size list of numbers, 64 in our models. A three-word chat message and a three-page email both have to be summarised into the same 64 numbers. For short inputs that is fine; for long ones the summary is necessarily lossy, however well the gates protect it. The gates decide *what* to keep; they cannot make the box bigger.

Limit 2 — it cannot look back. Once a word has been read, the only trace of it is whatever survived in the memory. There is no way for the model to re-read the input to check a detail — the way you would glance back at the date at the top of a letter before writing the reply. An RNN reads a ticket the way you would read it if it were shown to you one word at a time and then taken away.

Limit 3 — it reads strictly in order. Word 50 cannot be processed until words 1 to 49 have been. That is not a limitation of memory or accuracy but of *time*: a graphics card with thousands of cores can work on thousands of things at once, and a recurrent layer gives it one thing at a time. Slide 16 measures this.

The bottom band makes the key observation. For a router that outputs one label, limit 1 is survivable. It stops being survivable the moment the **answer is a sequence too** — a translation, a summary, an address rewritten into a standard format. Those outputs contain many details, and all of them would have to pass through the one memory. That is what the next slide sets up.



SLIDE 6 OF 40 · BLOCK 02 · ATTENTION

WHEN THE ANSWER IS A SEQUENCE TOO

Two readers back to back: an encoder reads, a decoder writes

Two readers back to back: an encoder reads the input into one summary, a decoder writes the output one piece at a time from that summary.

When the output is a sequence, the standard 2014 design was to use two recurrent networks. The **encoder** is exactly Week 4's reader: it reads the input, one character here, and produces its final memory — the orange *summary* box. The **decoder** is a second reader turned into a writer: it starts from the summary and produces the output one character at a time, feeding each character it writes back in as the next input, the way Week 4's next-word writer did. This is the *sequence-to-sequence* model of Sutskever, Vinyals and Le (2014), and, with a GRU, of Cho and colleagues in the same year.

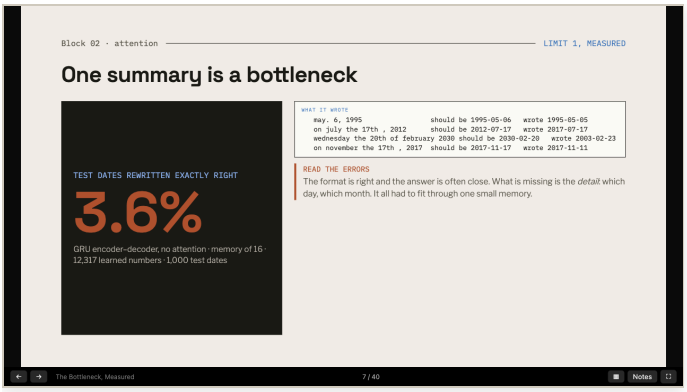
The task is chosen to be small enough to train in seconds and honest enough to expose the design. Six thousand training dates and a thousand test dates are written in a dozen formats — *received on 12 december 1994*, *jan 20 2012*, *wednesday the 20th of february 2030* — and the target is always the ten-character ISO form, `YYYY-MM-DD`. Two features matter. The output **reorders** the input: the year is usually written last but comes first in the standard form, so the decoder needs a detail from the far end of the input before anything else. And the details are unforgiving: a day of *17* written as *11* is simply wrong, whereas a translation can be a little off and still pass.

Look at the diagram's shape and notice where the bottleneck sits. Every arrow from the encoder's side to the decoder's side passes through one box. The encoder produced a memory after every character (the grey row); the decoder is handed only the last one. Hold on to that discarded row; the deck comes back for it. That is a design decision, and the next slide prices it.

One detail of the training, for readers of the code: during training the decoder is shown the *correct* previous character at each step rather than its own guess (the comment "training shows the decoder the correct previous character; testing uses its own guess"). This is called *teacher forcing*, and it is standard; at test time the model is on its own.

GO FURTHER

- Sutskever, Vinyals & Le, "Sequence to Sequence Learning with Neural Networks" (2014) — the design on this slide; §3.3 admits the bottleneck with an odd trick.
- Week 4 data README — how the date files were made.



SLIDE 7 OF 40 · BLOCK 02 · ATTENTION

LIMIT 1, MEASURED

One summary is a bottleneck

A GRU encoder–decoder with a 16-number memory gets 3.6% of test dates exactly right; its errors have the right shape and the wrong details.

This is the first measurement of the week and it is deliberately stark. A GRU encoder–decoder with a memory of 16 numbers, trained for ten passes over the 6,000 training dates, rewrites **3.6%** of the 1,000 test dates exactly right. “Exactly right” means all ten output characters correct; one wrong digit counts as a failure, which is the standard a date needs.

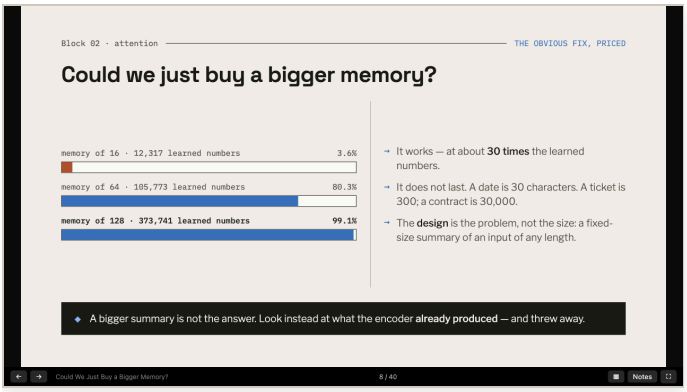
The instructive part is *how* it fails, so read the five examples. *on july the 17th , 2012* becomes *2017-07-17*: the year picked up the day’s digits. *wednesday the 20th of february 2030* becomes *2003-02-23*. In every case the format is right — four digits, a dash, two, a dash, two — and the month is usually right, but the specific digits have been smeared. That is what a bottleneck looks like from the outside. The gist of the input survives the one small memory; the details do not. The decoder is writing from a blur.

It is worth naming what is *not* the cause. It is not too little data: the same data will train the attention model to 97.6%. It is not too little training: the attention model gets the same ten epochs. It is not the format: the format is learned. The cause is that the whole date had to pass through 16 numbers, and a date has more than 16 numbers’ worth of detail in it. The 2014 sequence-to-sequence paper ran into exactly this, and reported that *reversing the source sentence* helped — a trick that only makes sense if the last thing the encoder read is the best-remembered, which is a confession that the memory is a bottleneck.

The natural response is the next slide’s question: if the memory is too small, make it bigger.

GO FURTHER

- Sutskever et al. (2014), §3.3 “Reversing the Source Sentences” — the trick, and the limitation it reveals.



SLIDE 8 OF 40 · BLOCK 02 · ATTENTION

THE OBVIOUS FIX, PRICED

Could we just buy a bigger memory?

Widening the memory from 16 to 128 fixes these dates — at about thirty times the learned numbers, and only until the inputs get longer.

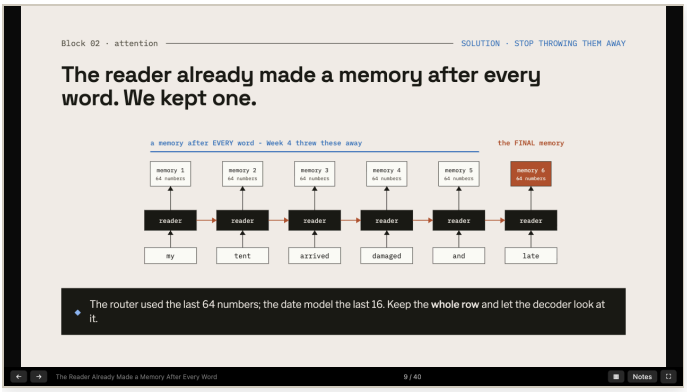
The honest answer to “could we just buy a bigger memory?” is yes, for now. The notebook retrains the same no-attention model with memories of 64 and 128 numbers. At 64 it reaches 80.3%; at 128 it reaches 99.1%. So the problem can be bought off.

Look at the price column. The 16-memory model has 12,317 learned numbers; the 128-memory model has 373,741, about **thirty times** as many. The growth is faster than the memory width because a recurrent layer’s parameters scale with the width *squared* (every memory number connects to every other), so doubling the memory roughly quadruples the layer.

Then look at why the fix does not last. A date is about thirty characters. A support ticket is a few hundred words; a contract is tens of thousands. A memory that is comfortably big enough for a date is a bottleneck again for a paragraph, and the same argument says you would need to widen it again — quadratically. The problem is not the size of the memory. It is the **design**: a fixed-size summary of an input of any length. No fixed size is right for all lengths.

The bottom band points at the turn. A bigger summary is not the answer; the encoder already produces a memory after every character, and we have been throwing all but the last away. The next slide makes that visible, and the one after shows how to use a whole row of memories at once: attention, in the smallest possible example.

A note on how to read experiments like this one throughout the course: three models, identical except for one setting, same data, same seed, same training budget. When one thing changes and the result moves, you know what moved it. That discipline is what makes the 3.6% → 97.6% comparison two slides later mean something.



SLIDE 9 OF 40 · BLOCK 02 · ATTENTION

SOLUTION · STOP THROWING THEM AWAY

The reader already made a memory after every word. We kept one.

The turn in the argument. Every recurrent layer hands back a memory after every word as well as the final one; the router and the date model kept only the final one. Keep the row.

The last two slides said that one summary is a bottleneck and that a bigger summary is a bad bargain. This slide is the turn: look again at what the reader was producing all along. The notebook cell runs the same twelve-word sentence through an RNN, a GRU and an LSTM and prints the shapes of what comes back. Every reader returns **two** tensors: the memory after every word, shape (1, 12, 64), twelve memories of 64 numbers each, and the final memory, shape (1, 1, 64).

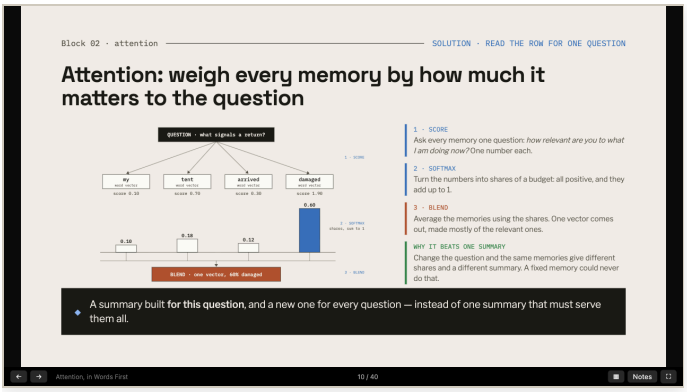
Week 4's router used only the second. It read a whole ticket, took the last 64 numbers, and classified from those. The date model on the previous slides did the same with 16: the encoder read the whole date, one character at a time, and only its final memory crossed to the decoder. In both cases the discarded material, the blue row in the diagram, was a memory after every input, each one the model's summary *up to that point*, and each one already computed. Nothing new has to be built to use them. What changes is what is kept.

That is the whole of attention, stated before its mechanism: hand the decoder the entire row instead of the last entry, and let it decide at every step of its writing which memories matter right now. The next slide is the mechanism, on exactly the four words drawn here: how to look at a whole row of memories at once, for one question, and come back with one vector.

A small technical footnote for anyone reading the code: the LSTM returns its final memory as a pair (the hidden state and the protected cell state), which is why the notebook takes the first element for the LSTM. The GRU and RNN return one tensor.

GO FURTHER

- PyTorch `nn.GRU` documentation — see the *Outputs* section: `output` is the memory at every step, `h_n` the final one.



SLIDE 10 OF 40 · BLOCK 02 · ATTENTION

SOLUTION · READ THE ROW FOR ONE QUESTION

Attention: weigh every memory by how much it matters to the question

Attention, before any arithmetic: ask every memory how relevant it is to the question, turn the answers into shares of a budget, and average the memories by those shares. A summary built for one question.

This slide states the idea in words and a picture before the next slide does the sums. The decoder has a row of memories in front of it and is about to write something; it has a **question** in mind. Week 4's design let it see one summary and hope the answer was in there. Attention lets it read the whole row, for this question, in three moves.

Score. Ask every memory one question: *how relevant are you to what I am doing now?* The answer is one number per memory. In the picture the question is *what signals a return?* and the four words of a ticket are the memories; *damaged* scores highest, *my* lowest.

Softmax. Turn the scores into **shares of a budget**. Every share is positive, and the shares add up to exactly 1. The bars are those shares: *damaged* takes about 60% of the budget, *tent* 18%, *arrived* 12%, *my* 10%. The sum-to-1 property is the one to remember. Attention is a budget: spending more on one memory means spending less on the others, which is why the shares are readable as "where the model looked" later on.

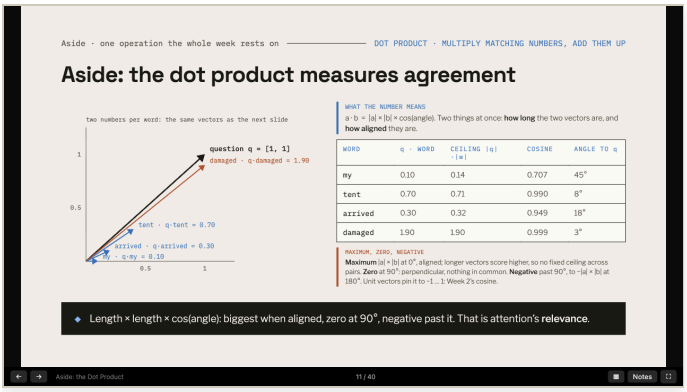
Blend. Average the memories using the shares. One vector comes out, made mostly of the relevant memories and a little of everything else. That vector is what the decoder gets instead of a fixed summary.

Why this beats one summary. Nothing about the memories changed. Change the question and the scores change, so the shares change, so the blend changes: the same row of memories yields a different summary for every question asked of it. A fixed memory cannot do that, and that is the whole of what attention adds. In Block 3 the question gets the name *query*, the memories being scored become *keys*, and what is blended becomes *values*, but the three moves are exactly these.

The numbers in the picture are real: they come from the next slide's arithmetic, done on two-number word vectors in the notebook, so you can check every one of them by hand.

GO FURTHER

- Grant Sanderson (3Blue1Brown), "Attention in transformers, visually explained" — the same three moves animated, in the vocabulary Block 3 will use.
- Lilian Weng, "Attention? Attention!" (2018) — a compact survey that starts from exactly this three-step view.



SLIDE 11 OF 40 · WRAP-UP

DOT PRODUCT · MULTIPLY MATCHING NUMBERS, ADD THEM UP

Aside: the dot product measures agreement

An aside on the one operation the score uses, and half of what this course computes: multiply matching numbers, add them up, and read the result as how much two vectors agree.

The previous slide said “ask every memory how relevant it is to the question” and gave each one a score. This aside is what that score actually is, because the same operation runs through the whole course under different names. The **dot product** of two vectors of the same length: multiply the numbers that sit in the same position, then add the products. Two vectors in, one number out.

The picture uses the four word vectors and the question from the next slide, drawn as arrows in the plane. The question is [1, 1], the arrow at forty-five degrees. *Damaged* is [1.0, 0.9]: long, and pointing almost the same way, so its dot product with the question is 1×1.0 + 1×0.9 = 1.90. *My* is [0.1, 0.0]: short, and pointing along one axis only, so its dot product is 0.10. *Tent* and *arrived* fall in between. The box on the right does all four multiplications in the open.

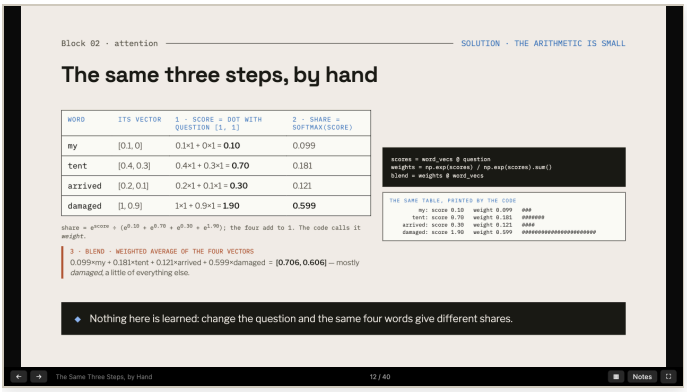
What the number means. The dot product equals the two lengths multiplied together, times the cosine of the angle between the vectors: $a \cdot b = |a| \times |b| \times \cos\theta$. So one number carries two facts at once, how long the vectors are and how aligned they are. That gives every pair a ceiling: the most a and b can score is $|a| \times |b|$, reached when the angle is 0° and they point exactly the same way. The table on the slide shows how close each word comes. *Damaged* scores 1.90 against a ceiling of 1.90: it is aligned with the question to within about three degrees. *My* scores 0.10 against a ceiling of 0.14, at forty-five degrees. There is no fixed maximum across different pairs, because a longer vector can always score higher; that is exactly why Week 2’s cosine similarity divides the lengths back out, so that only direction remains and every pair lives between −1 and 1.

Ninety degrees, and negative. At 90° the cosine is zero, so the dot product is zero whatever the lengths: perpendicular vectors have no component along each other, and in this space that means “nothing in common”. Past 90° the cosine turns negative, and so does the dot product, down to $-|a| \times |b|$ at 180°, when the vectors point in opposite directions. So yes, it can be negative. In attention a negative score does not switch a memory off; softmax turns it into a small positive share, never exactly zero. Alignment, then, is the top of the scale, 90° is the middle where the two vectors are simply unrelated, and opposition is the bottom.

Where you have already met it. Week 2’s document similarity is this, with the lengths divided out. Week 3’s classifier score, weights times features summed, is this. The linear layer at the top of every transformer that turns a vector into one score per token is this, once per token. When a slide later says a model “compares”, “scores” or “matches”, it is multiplying matching numbers and adding them up.

GO FURTHER

- Week 2 lecture notebook — dot product and cosine on real document vectors.
- Grant Sanderson (3Blue1Brown), “Dot products and duality” — the geometric meaning, in fourteen minutes.



SLIDE 12 OF 40 · BLOCK 02 · ATTENTION

SOLUTION · THE ARITHMETIC IS SMALL

The same three steps, by hand

The three steps with numbers small enough to check by hand: four words, two numbers each, one question vector, and the same table printed by three lines of code.

Here are the previous slide's three moves as arithmetic, small enough for a pocket calculator. Four words of the ticket *my tent arrived damaged*, each a two-number vector (*my* is [0.1, 0.0], *damaged* is [1.0, 0.9]), and a question vector, [1, 1], standing for "what signals a return?". The table on the slide does every step in the open.

Step 1, score. Compare the question with each word's vector using a dot product: multiply matching numbers and add. This is the same operation behind Week 2's cosine similarity, minus the normalisation: a large positive score means the two vectors point the same way. Here the scores are 0.10 for *my*, 0.70 for *tent*, 0.30 for *arrived*, 1.90 for *damaged*. The table shows the multiplication for each row.

Step 2, softmax. Turn the scores into weights. Softmax exponentiates every score and divides by the total, which does two useful things: every weight is positive, and the weights add up to exactly 1. The scores become 0.099, 0.181, 0.121 and 0.599, the *share* column of the table: share = e^score ÷ (e^0.10 + e^0.70 + e^0.30 + e^1.90). The code prints the same numbers under the name *weight*, and the previous slide drew them as bar heights; share, weight and attention weight are one quantity. Notice that softmax exaggerates differences — the biggest score gets most of the budget — but never gives anything a weight of exactly zero. It is a *soft* version of picking the maximum, hence the name.

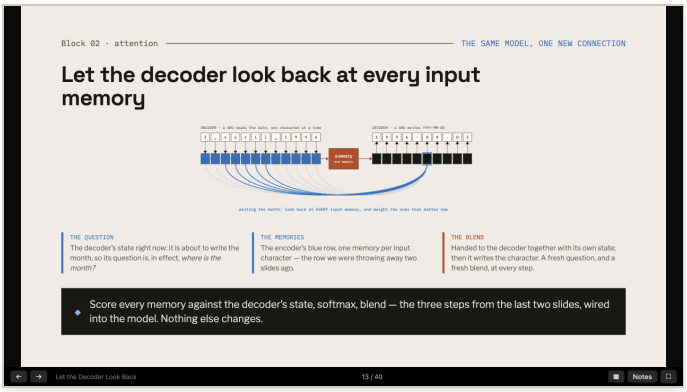
Step 3, blend. Take a weighted average of the word vectors using those weights. The result is one vector, mostly *damaged* with a little of everything else: [0.706, 0.606].

The sum-to-1 property is the one to remember. It makes attention a **budget**: the model has exactly one unit of attention to spend across the words, and spending more on one word means spending less on the others. That is why attention weights are readable as "where the model looked" (slide 15), and why "every row sums to 1" keeps coming back.

The right-hand column is the same three steps as three lines of code, and the printout under it is the table again. Nothing on this slide is learned: change the question and the shares change with it. Try [1, −1]: every score comes out 0.10, the four shares are 0.25 each, and the blend is the plain average, which is what attending to nothing in particular looks like. In a trained model the question and the word vectors come from learned parameters, but the shares are always computed fresh for each input. Two different things are both called *weights*; keep them apart.

GO FURTHER

- Week 2 lecture notebook — dot product and cosine, which step 1 reuses.
- Lilian Weng, "Attention? Attention!" (2018) — a compact survey of attention variants, starting from exactly this three-step view.



SLIDE 13 OF 40 · BLOCK 02 · ATTENTION

THE SAME MODEL, ONE NEW CONNECTION

Let the decoder look back at every input memory

In the date model the question is the decoder's current state and the things scored are the encoder's memories at every input character — a fresh blend at every step of writing.

Now the three steps go into the encoder–decoder, and the diagram shows where. Nothing about the encoder changes: it still produces a memory after every input character. The change is entirely on the decoder's side. At each step of writing, the decoder's own current state plays the role of the **question**: "I am about to write the month — where is it?" That state is scored against every one of the encoder's memories (the blue boxes), the scores are softmaxed into weights, and the memories are blended by those weights into a *context* vector that is handed to the decoder alongside the character it wrote last. Then the decoder writes, its state moves on, and the next step asks a fresh question and gets a fresh blend.

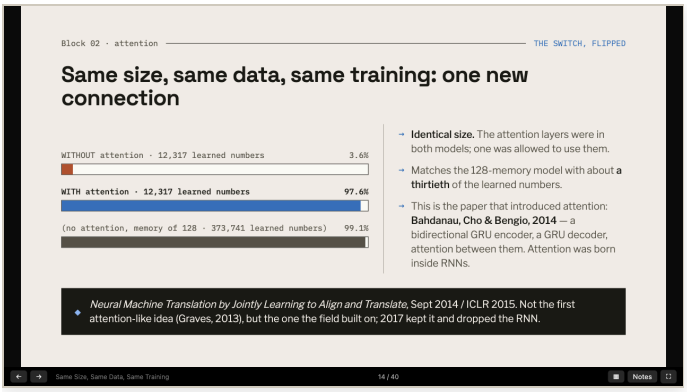
The curves in the diagram are one such step, the one writing the month. The thick lines go to the letters of *april*; the thin lines to everything else. That is the weight budget being spent where the answer is.

Three details from the code are worth knowing without needing to read it. First, the encoder here reads in both directions (a *bidirectional* GRU), so each input memory knows what came before and after that character — a cheap improvement that Bahdanau's original also used. Second, the scoring is a small learned function (two tiny `nn.Linear` layers and a `tanh`), not a raw dot product, so the model can learn *what kind* of match counts; the principle is unchanged. Third, padding positions are masked out before the softmax, so the model cannot spend budget on empty slots.

This is the design of Bahdanau, Cho and Bengio (2014), published within months of the plain encoder–decoder it improves. Their paper is in this week's readings as the place the idea came from; the mechanism on this slide is their §3.1.

GO FURTHER

- Bahdanau, Cho & Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate" (2014) — §1 states the bottleneck; §3.1 defines the weights.
- Olah & Carter, "Attention and Augmented Recurrent Neural Networks" (Distill, 2016) — animated version of this exact diagram.



SLIDE 14 OF 40 · BLOCK 02 · ATTENTION

THE SWITCH, FLIPPED

Same size, same data, same training: one new connection

Same data, same training, identical learned numbers — one new connection takes the model from 3.6% to 97.6%.

This is the week's cleanest experiment, and it is worth spelling out why it is clean. The notebook's `DateRewriter` has a switch, `use_attention`. Both models are built from the same class, so the attention layers exist in both; with the switch off they are simply never used. Both are trained from the same random start (same seed), on the same 6,000 dates, for the same ten epochs, with the same optimiser settings. The learned-number count is identical to the digit: 12,317. The **only** difference is whether the decoder is allowed to look back at the encoder's memories.

With the switch off: 3.6% of test dates exactly right. With it on: **97.6%**. That is the whole case for attention in one bar chart.

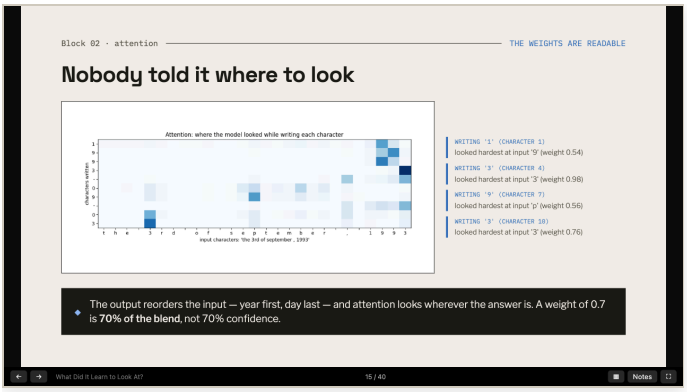
The grey bar is the comparison that gives the result its scale. The bigger-memory fix reached 99.1%, but needed 373,741 learned numbers to do it. Attention gets within a point and a half with a **thirtieth** of the numbers, and, unlike the bigger memory, it does not stop working when the inputs get longer: the decoder blends however many memories the encoder produced.

Where attention came from. This is the design of the paper that introduced attention for neural machine translation: Bahdanau, Cho and Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, posted in September 2014 and presented at ICLR 2015, which is why you will see both dates cited. It was an RNN paper. Its encoder was a bidirectional GRU producing one representation per source word; its decoder was a GRU generating the translation one word at a time; and its attention mechanism selected a weighted combination of the encoder representations at every output step, to escape exactly the fixed-length bottleneck the previous slides measured. Our date model is that design in miniature, down to the additive scoring in the appendix code. Two qualifications. It was not the first attention-like mechanism in neural networks; the paper itself discusses Graves's 2013 handwriting-generation model. And the 2017 Transformer paper did **not** introduce attention: its contribution was an architecture built around attention with no recurrence, which is where Block 3 goes.

Historically this design was the 2014–2016 state of the art. An RNN encoder–decoder with attention is what Google Translate switched to in November 2016 (the "GNMT" system: LSTMs, eight layers deep, with attention between encoder and decoder), and it cut translation errors by a reported 60% against the previous phrase-based system. It is also, as the next slide shows, a model whose reasoning you can partly read.

GO FURTHER

- Bahdanau, Cho & Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate” (2014) — the paper that introduced attention; §3 is the model on this slide.
- Graves, “Generating Sequences With Recurrent Neural Networks” (2013) — the earlier attention-like mechanism Bahdanau et al. cite.
- Wu et al., “Google’s Neural Machine Translation System” (2016) — the production system built on this design.
- Google Research blog, “A Neural Network for Machine Translation, at Production Scale” (2016) — the announcement, for the non-technical account.



SLIDE 15 OF 40 · BLOCK 02 · ATTENTION

THE WEIGHTS ARE READABLE

Nobody told it where to look

The attention weights are readable: for each character written, they show how much the model looked at each input character — and nobody told it where to look.

The heatmap is the attention weights for one test input, *the 3rd of september , 1993*, which the model rewrites correctly as *1993-09-03*. Rows are the ten characters written; columns are the input characters; darker means more of the attention budget spent there. Each row sums to 1.

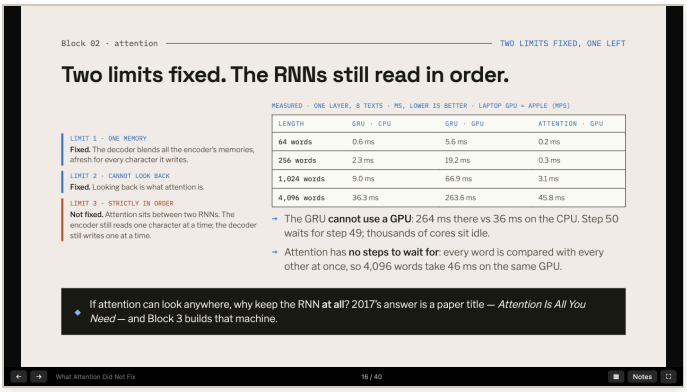
Read it row by row. Writing the first *1* of the year, the model looks hardest at the *9* near the end of the input (weight 0.54), where the year is. Writing the *3* of *1993* it looks at the *3* in *1993* with weight 0.98 — almost the whole budget on one character. Writing the *9* of the month *09*, it looks at *p* in *september* (0.56): it has learned that the month's identity lives in the month's name. Writing the final *3* of the day, it looks at the *3* of *3rd* (0.76), near the front of the input.

Two observations. First, the output order and the input order are different — the year is last in the input and first in the output — and attention does not care; it looks wherever the answer is, regardless of position. That is precisely what a fixed final memory could not do. Second, and this is the deeper point: **nobody told the model which input characters matter**. The only supervision was the correct output string. The alignment between output characters and input characters was learned as a by-product of getting the answer right, because looking in the right place is what makes the answer right. Bahdanau's paper has the same figure for French–English translation (their Figure 3), with words instead of characters, and it made the same impression in 2014.

Attention weights are one of the few parts of a neural network you can inspect and interpret directly. They are not a complete explanation of what the model is doing — a weight says where the model looked, not what it did with what it found — but they are a genuine window, and slide 38 opens the same window on a real language model.

GO FURTHER

- Bahdanau et al. (2014), Figure 3 — the original alignment heatmaps, French to English.



SLIDE 16 OF 40 · BLOCK 02 · ATTENTION

TWO LIMITS FIXED, ONE LEFT

Two limits fixed. The RNNs still read in order.

Attention fixed the one memory and the inability to look back. It did not fix reading one step at a time — the limit that matters at scale. So: do we need the RNN at all?

Check the three limits of slide 5 off. Limit 1, one memory: fixed — the decoder blends all the encoder's memories, afresh at every step. Limit 2, cannot look back: fixed — looking back is exactly what attention is. Limit 3, strictly in order: **not fixed**. The encoder is still a GRU that reads one character at a time, and the decoder is still a GRU that writes one character at a time. Attention sits between them; it does not change how either of them works.

The table is the measurement: one GRU layer and one self-attention layer, both with 64-number vectors, timed on a batch of eight texts as the length grows from 64 to 4,096 words. The slide shows the GRU on the CPU and on the laptop's GPU (Apple's MPS) and attention on the GPU; the notebook prints attention on the CPU as well. Two patterns appear.

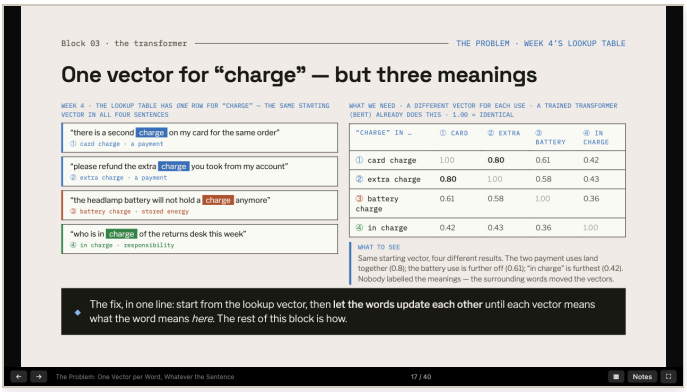
The **GRU's time grows in step with the length**, and the GPU makes it *slower*, not faster: each step needs the result of the step before it, so thousands of cores wait their turn.

The **attention layer behaves in the opposite way**: fast on the GPU, because every position is handled at once in one matrix multiplication. Its own cost is in the slide's small print: every word is compared with every other, so four times the words is about **fifteen** times the time (3.1 → 45.8 ms on the GPU; the theory says sixteen, the square of four). On the CPU that bill is already due: at 4,096 words the attention layer (86 ms) is slower than the GRU (36 ms). That square-of-the-length cost is what makes long context windows expensive; slide 28 and Week 7 return to it.

So the question in the bottom band. By 2016 attention was doing the heavy lifting in the best translation systems, and the recurrence around it was the slow part, and the long way round: word 1 reaches word 500 only by surviving 499 rewrites of the memory, where attention connects any two positions in one step. If attention can look anywhere, do we need the RNN at all? The question was asked in June 2017 by a group at Google Brain and Google Research, in a paper whose title is the answer: *Attention Is All You Need*. Their proposal, the **transformer**, keeps attention, removes the recurrent networks entirely, and adds one ingredient to replace what the recurrence had been supplying for free: information about where each word sits. Block 3 builds that machine one idea at a time, in the order the problems arise. What made the design take over everything else is the property measured on this slide: nothing in it waits for anything else, so it could be made larger, and trained on more, than any recurrent model.

GO FURTHER

- Vaswani et al., "Attention Is All You Need" (2017) — the required paper; §4 is this slide's table, Table 2 the training cost.
- Jay Alammar, "The Illustrated Transformer" (2018) — the required pictures for §3.



SLIDE 17 OF 40 · BLOCK 03 · THE TRANSFORMER

THE PROBLEM · WEEK 4'S LOOKUP TABLE

One vector for "charge" — but three meanings

Week 4's lookup table gives the word "charge" one vector, whatever the sentence; the four tickets on the slide use it in three senses. A trained transformer gives it a different vector in each — and building that is what the rest of this block does.

Block 3 opens with the problem it has to solve, stated in Week 4's own terms. The embedding table you built last week is a lookup: the word *charge* has one row, and that row is what the model sees every time the word appears. The four numbered sentences on the left use it in three senses. In ① *a second charge on my card* and ② *refund the extra charge* it means a payment, with different words around it each time; in ③ *the battery will not hold a charge* it means stored energy; in ④ *who is in charge of the returns desk* it means responsibility. A router that cannot tell those apart will send two of the three kinds of ticket to the wrong queue. The problem is not that the table is poorly trained. It is that **one vector per word cannot represent a word whose meaning depends on its neighbours**.

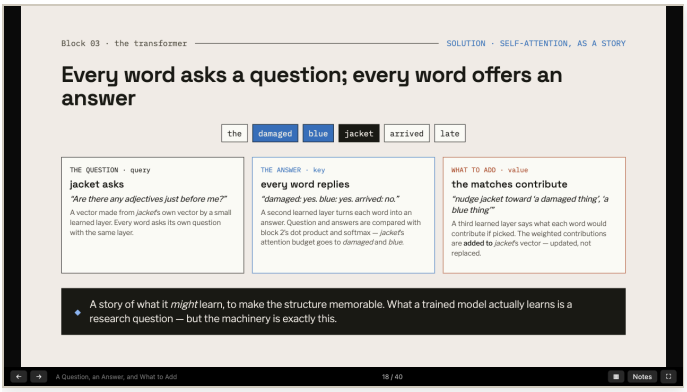
The right-hand table is the proof that a fix exists, and its rows and columns carry the same numbers ①–④ as the cards. It comes from BERT, a trained transformer that Block 4 discusses properly. The notebook takes the same starting row for *charge* — one row, 768 numbers — runs each sentence through the model, and pulls out the vector for *charge* **after** the blocks. Then it compares the four vectors with cosine similarity, Week 2's measure and the geometry of the dot-product aside. The two payment uses, ① and ②, land at 0.80. The battery use, ③, sits at about 0.6 from either of them. *In charge*, ④, is furthest of all, around 0.4. Nobody labelled those meanings. The model was never told that *card* and *refund* signal money or that *battery* signals energy; the surrounding words did the work, through the mechanism the next slides describe.

This is rung 3 of the ladder Week 4 drew: counts (Week 2) → one static vector per word (Week 4) → **a vector per word, in context** (this week) → embeddings as a service (Week 7). It is also the answer to a question you may have had since Week 4: if word vectors are so useful, why did the router still need the LSTM? Because a static vector for *charge* is a compromise across all its meanings, and only context can resolve it.

The bottom band states the fix in one line: let the words **update each other** until each vector means what the word means *here*. Three slides show how.

GO FURTHER

- Week 5 notebook, §1.0 — the cell that produces the table; change the sentences and see how the numbers move.
- Peters et al., "Deep contextualized word representations" (2018) — the ELMo paper, the first to make "a vector per word, in context" a standard tool; its §5.3 has the same one-word-three-meanings demonstration.



SLIDE 18 OF 40 · BLOCK 03 · THE TRANSFORMER

SOLUTION · SELF-ATTENTION, AS A STORY

Every word asks a question; every word offers an answer

Every word asks a question of every other word, every word offers an answer, and the words that match add something to the asker. Told as a story, with an honest caveat.

In Block 2 the *question* came from the decoder and the things being scored came from the encoder — two different networks. Remove the RNNs and there is nobody to ask. The transformer's answer is to let **every word ask its own question of every other word in the same sequence**. That is self-attention, and this slide tells it as a story so the three parts have names you can hold on to.

Take *the damaged blue jacket arrived late*. Every word is turned into three vectors by three small learned layers — the same three layers for every word.

The question — the query. *Jacket*'s query vector might amount to "are there any adjectives just before me?" It is made from *jacket*'s own vector, so a noun's query is a noun's kind of question.

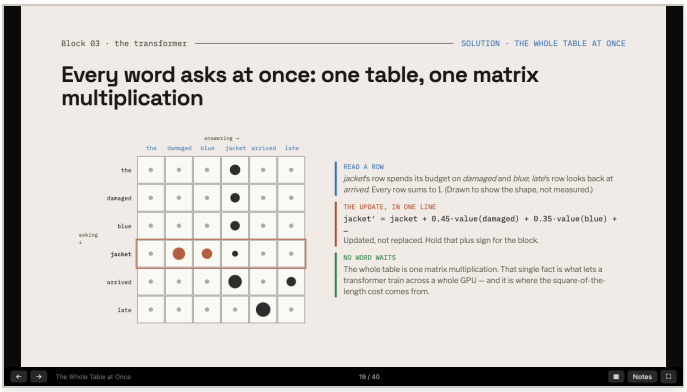
The answer — the key. Every word produces a key: a vector that says what it has to offer. *Damaged* and *blue* produce keys that match *jacket*'s query — in vector terms, that point the same way, so the dot product is large. *Arrived* and *late* produce keys that do not. Matching is Block 2's score-and-softmax, unchanged: *jacket*'s attention budget goes mostly to *damaged* and *blue*.

What to add — the value. Each word also produces a value: the contribution it would make if it were picked. *Damaged*'s value nudges *jacket*'s vector toward "a damaged thing"; *blue*'s toward "a blue thing". The weighted values are **added** to *jacket*'s own vector. Out comes a vector that means *a damaged blue jacket* — the vector-in-context that the previous slide showed a trained model producing.

Two things the story is careful about. First, the caveat in the bottom band: this is a story of what a head *might* learn, chosen to make the structure memorable. Whether a particular head in a particular trained model really tracks adjectives is a research question (some do something like it; most are less tidy). The **machinery**, however, is exactly this — three learned layers, then score, softmax, blend — and that is what the next slide computes. Second, notice that every word runs this at the same time. *Late* asks its own question ("what event am I describing?") with the same three layers, and gets *arrived* back. Nothing waits for anything.

GO FURTHER

- Grant Sanderson (3Blue1Brown), "Attention in transformers, visually explained" — the source of this way of telling it, with the adjective–noun example animated.
- Jay Alammar, "The Illustrated Transformer", the "Self-Attention in Detail" section — the same three vectors drawn step by step.



SLIDE 19 OF 40 · BLOCK 03 · THE TRANSFORMER

SOLUTION · THE WHOLE TABLE AT ONCE

Every word asks at once: one table, one matrix multiplication

Every word asks at once, so the result is a table: rows ask, columns answer, each row sums to 1. The whole table is one matrix multiplication, and the update is an addition.

Do the previous slide for every word simultaneously and you get a **table**, which researchers call the attention pattern. Rows are the words asking; columns are the words answering; the size of each dot is the share of the asker's budget spent on that word. Every row sums to 1, because every row is a softmax.

This particular table is **drawn, not measured** — the slide says so — to show the shape you would hope for after training: *jacket*'s row spends its budget on *damaged* and *blue*; *late*'s row looks back at *arrived*; *the* has little to ask and spreads thinly. The appendix computes a real one, in four lines of code.

The right-hand column makes three points that carry the rest of the block.

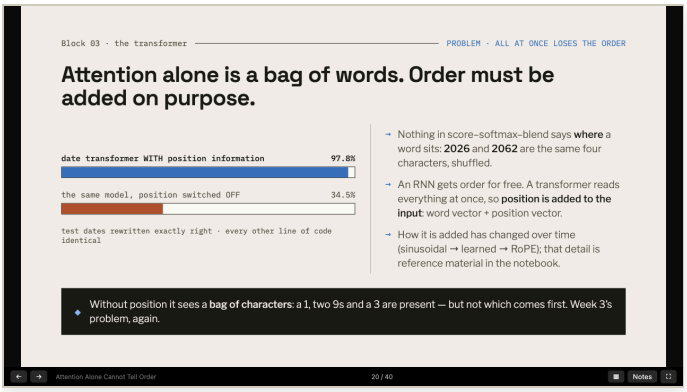
The update is an addition. *jacket′ = jacket + 0.45·value(damaged) + 0.35·value(blue) + …* The word's vector is not replaced by the blend; the blend is added to it. This matters more than it looks. It means a word can pass through many rounds of attention and still be recognisably itself, with information accumulated on top, and it is the "skip connection" you will see drawn around every part of the block on slide 21. Hold the plus sign.

No word waits for another. Every query is scored against every key in one matrix multiplication — the operation that GPUs do fastest — so the whole table appears at once. That is the fix for limit 3, and the reason a transformer can be trained on hardware that an RNN mostly leaves idle.

And it is where the cost comes from. A table for n words has n^2 entries. Double the text, quadruple the table. Slide 16 measured this; Block 5 and Week 7 will pay for it.

GO FURTHER

- Alammar, "The Illustrated Transformer", the matrix-form section — the same table written as $Q \cdot K^T$.



SLIDE 20 OF 40 · BLOCK 03 · THE TRANSFORMER

PROBLEM · ALL AT ONCE LOSES THE ORDER

Attention alone is a bag of words. Order must be added on purpose.

Nothing in score–softmax–blend mentions where a word sits, so a transformer is order-blind unless order is added on purpose. Measured: 97.8% with position, 34.5% without.

An RNN never needed to be told the order of the words, because reading in order is how it computes: its memory after word five exists only because it read words one to four first. Self-attention has no such structure. Look back at the four lines on slide 37 — nothing in them refers to a position. Every query is compared with every key by content alone, so if you shuffled the words of the input, you would get exactly the same output vectors back, shuffled. To a transformer with nothing else, *dog bites man* and *man bites dog* are the same bag of words. That is Week 3's bag-of-words wall, rebuilt inside a far more powerful model.

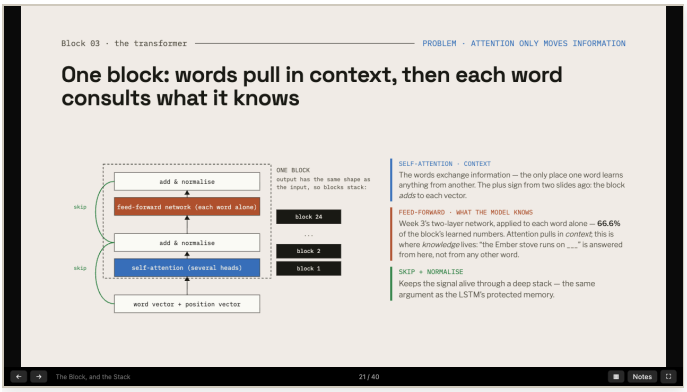
The fix is to **add position to the input**. Each position gets its own vector — position 1, position 2, and so on — and that vector is added to the word's vector before the first block. A word's vector then carries two things, what the word is and where it sits, and the attention layers can use either. (How the position vectors are made has changed over the years — fixed sine waves in 2017, learned vectors in BERT, rotations applied inside attention in most current models. The notebook covers the three as *reference* material; the idea above is what you need.)

The bars are the measurement. The date transformer of Block 3 (its code is on slide 37) scores 97.8% with its position vectors. Retrained with position switched off — every other line of code identical — it scores **34.5%**. Without position the model knows that a *1*, two *9*s and a *3* are present in the input, but not which comes first, and a date is nothing but order.

The general lesson, which you will meet again in Week 7 when documents are chopped into pieces and re-assembled: a transformer knows only what is in its input vectors. Anything you want it to use — including something as basic as word order — has to be put there.

GO FURTHER

- Vaswani et al. (2017), §3.5 "Positional Encoding" — the original sine-wave scheme, and why they chose it.
- Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021) — RoPE, the method most current models use; §3 for the idea, not the proofs.



SLIDE 21 OF 40 · BLOCK 03 · THE TRANSFORMER

PROBLEM · ATTENTION ONLY MOVES INFORMATION

One block: words pull in context, then each word consults what it knows

Attention only moves information between words. A block adds the part that holds knowledge — a feed-forward network applied to each word alone — wraps both in skip connections, and stacks.

Attention lets words exchange information, and that is *all* it does. Two things are still missing. Somewhere for the model's **knowledge** to live: ask a model to complete *the Ember camp stove runs on* ___ and the answer is in no other word of the sentence; it was learned during training and has to be stored somewhere. And a way to repeat the attention step many times without the signal degrading on the way. The transformer **block** supplies both.

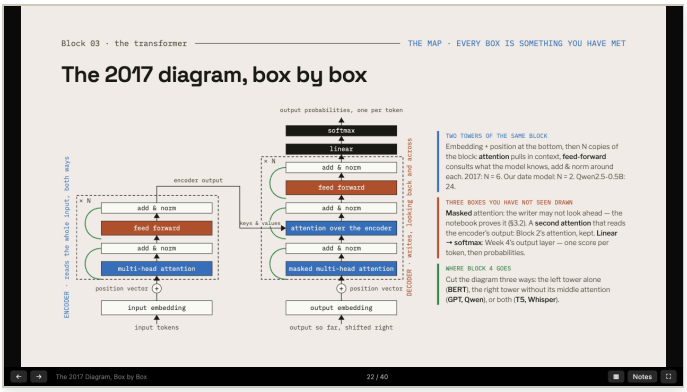
Read the diagram bottom to top. The input is each word's vector plus its position vector. Then **self-attention** (several heads): the words pull in context. Then an *add & normalise* step. Then a **feed-forward network** — Week 3's two-layer network — applied to each word *on its own*, with no view of the other words: each word, having gathered its context, thinks. Then another *add & normalise*. The output has exactly the shape of the input, so a block's output is the next block's input, and blocks **stack**: the small model on slide 38 has 24 of them, and large models have around a hundred.

Two facts about the block that people consistently get backwards. **Most of the learned numbers are in the feed-forward part, not in attention.** In the block the notebook builds, 66.6% of the parameters are feed-forward, 33.3% attention, 0.1% normalisation, because the feed-forward layer expands each vector to four times its width and back. It is also where researchers find stored facts when they go looking. A fair one-line summary of the block, then: attention pulls in **context**; the feed-forward network consults what the model **knows**.

The skip connections are the plus sign from slide 19. Each part of the block *adds* its result to the vector that came in, rather than replacing it. That is what makes a stack of 24 or 96 blocks trainable: the original signal always has an uninterrupted path to the top, so a word's vector at block 20 is still recognisably that word, with twenty rounds of context and knowledge accumulated on top. The Week 4 LSTM's protected memory was the same idea in a different costume.

GO FURTHER

- Grant Sanderson (3Blue1Brown), "How might LLMs store facts" — the feed-forward layer as a fact store, animated.
- Meng et al., "Locating and Editing Factual Associations in GPT" (2022) — the evidence that facts live in the feed-forward layers; read the introduction.
- Alammar, "The Illustrated Transformer" — "The Residuals" section draws the skip connections.



SLIDE 22 OF 40 · BLOCK 03 · THE TRANSFORMER

THE MAP · EVERY BOX IS SOMETHING YOU HAVE MET

The 2017 diagram, box by box

The diagram that opens every introduction to transformers, redrawn so that every box is something you have already built.

This is Figure 1 of *Attention Is All You Need* (Vaswani et al., 2017), the picture in every textbook, blog post and video about transformers, redrawn in this deck's vocabulary. If it has looked intimidating, the point of this slide is that you have now met every box in it.

Read each tower from the bottom. Tokens go into an embedding table, Week 4's lookup table, and each word's vector has its position vector added (the ⊕, slide 20). Then N copies of the block from the previous slide: multi-head attention, add & norm, feed forward, add & norm, with a skip path around each part. The dashed box and the "× N" say *repeat*: N = 6 in the paper, 2 in the date model on the next slide, 24 in Qwen2.5-0.5B.

The decoder tower differs in two places, and you know both. Its self-attention is *masked*: each position may look only at earlier ones, because a writer must not be shown the word it is about to predict; the notebook (§3.2) proves that a trained decoder obeys this exactly. And it has a *second* attention whose question comes from the decoder and whose keys and values come from the encoder's output: Block 2's attention (slide 13), kept exactly. The label "output so far, shifted right" is the training trick behind the causal mask (notebook §3.2): the decoder's input is the target moved one place, so position t sees tokens 1 to $t-1$ and is scored on token t.

The head. A linear layer turns the final vector at each position into one score per token in the vocabulary, and a softmax turns the scores into probabilities. That is Week 4's output layer, unchanged; the generation loop of slide 28 picks the next token from it.

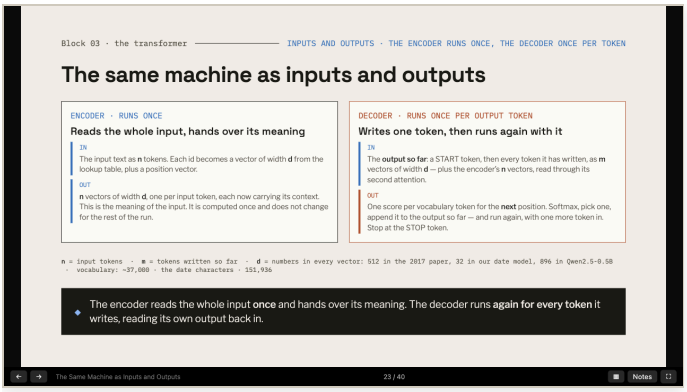
What is not in it. No recurrence and no convolution; that is the paper's title. One detail has changed since: the figure normalises *after* each sub-layer, while nearly every model today, including the notebook's block, normalises *before*, which trains more easily.

Sizes, for scale. The paper: N = 6 per tower, 512-number vectors, 8 heads, about 65 million learned numbers. The date model on the next slide: N = 2, 32-number vectors, 4 heads, 63,053 learned numbers, trained in 8 seconds.

Block 4 will cut this drawing three ways: the left tower alone is BERT; the right tower without its middle attention is GPT and every chat model since; both towers are T5 and Whisper.

GO FURTHER

- Vaswani et al., “Attention Is All You Need” (2017) — Figure 1 is this slide; §3.1–3.2 are now readable.
- Alammar, “The Illustrated Transformer” — the same figure unpacked one box at a time.
- Rush et al., “The Annotated Transformer” — the paper as runnable PyTorch, one box at a time.
- PyTorch, `nn.Transformer` — the class the appendix’s date model uses.



SLIDE 23 OF 40 · BLOCK 03 · THE TRANSFORMER

INPUTS AND OUTPUTS · THE ENCODER RUNS ONCE, THE DECODER ONCE PER TOKEN

The same machine as inputs and outputs

The 2017 diagram as inputs and outputs. The encoder runs once and hands over the meaning of the whole input; the decoder runs again for every token it writes, reading its own output back in.

The previous slide named the boxes. This one asks the two questions students most often cannot answer about the machine: what goes into each half and what comes out, and how many times each half runs. Three letters carry the answer. *n* is the number of tokens in the input; *m* is the number of tokens the decoder has written so far; *d* is the **width** of every vector, the count of numbers in it, fixed when the model is built: 512 in the 2017 paper, 32 in our date model, 896 in Qwen2.5-0.5B.

The encoder runs once. In: the input text as *n* tokens. Each token id is looked up in the embedding table and becomes a vector of width *d*, Week 4's lookup, and a position vector of the same width is added to it. Out: *n* vectors of width *d*, one per input token, each now carrying its context. That block of numbers is the meaning of the input as far as the model is concerned, and it is computed once. Nothing the decoder does afterwards changes it.

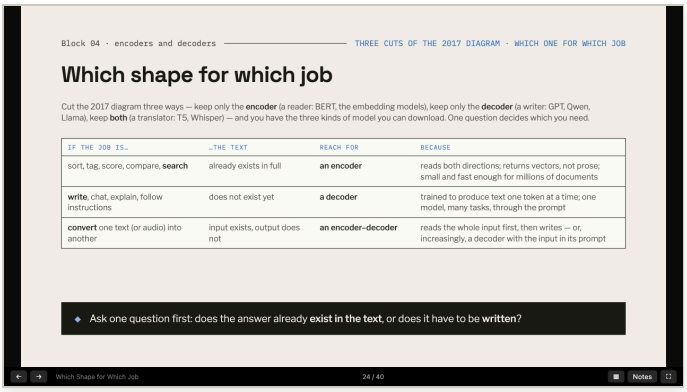
The decoder runs once per output token. In: the *output so far*, which starts as a single START token and grows by one token each time round, as *m* vectors of width *d*, plus the encoder's *n* vectors, which the decoder's second attention reads on every pass. Out: one score for every token in the vocabulary, for the next position only. Softmax turns the scores into probabilities, one token is picked (slide 27 is about how), it is appended to the output so far, and the decoder runs again with $m + 1$ tokens in. This continues until the model emits the STOP token. So an eight-token answer means one encoder pass and nine decoder passes, and the decoder's own output is literally its next input. In a decoder-only model such as Qwen there is no encoder at all: the prompt is simply the first part of the output so far, and the same loop does everything.

During training the loop is short-circuited. The decoder is shown the correct target shifted one place to the right, so every position predicts the next token from the correct earlier ones, and all positions can be scored in one pass; this is why the diagram labels its input "shifted right". The recursion is a property of generation, not of training.

Shapes. The two halves keep one vector per token, of width *d*, at every block. Only two steps change shape: the lookup at the bottom, which turns an integer into a *d*-vector, and the linear layer at the top, which turns a *d*-vector into one score per vocabulary token, 512 → about 37,000 in the paper and 896 → 151,936 in Qwen. The notebook prints those two Qwen numbers as (1, 4, 896) and (1, 4, 151936) for a four-token input.

GO FURTHER

- Alammar, “The Illustrated Transformer” — the “Decoder Side” section animates exactly this loop: encoder once, decoder step by step.
- Week 5 notebook, §5 — the by-hand generation loop that reproduces the library’s output token for token.



SLIDE 24 OF 40 · BLOCK 04 · ENCODERS AND DECODERS

THREE CUTS OF THE 2017 DIAGRAM · WHICH ONE FOR WHICH JOB

Which shape for which job

Ask one question first: does the answer already exist in the text, or does it have to be written? Sort, tag, compare, search → encoder. Write, chat, explain → decoder. Convert one text into another → encoder–decoder, or a decoder with the input in its prompt.

This is the slide to remember when someone in a meeting asks which model to use. The rule of thumb has three rows, and one question sorts almost every request into a row: **does the answer already exist in the text, or does it have to be written?**

If the job is to sort, tag, score, compare or search, the text already exists in full and the job is to understand it. Reach for an **encoder**. It reads both directions, returns vectors and scores rather than prose, and is small and fast enough to run over millions of documents. Classifying tickets into queues, finding duplicates, ranking search results, flagging abusive messages: encoder jobs. Using a chat model for these works but is like hiring a novelist to sort the mail.

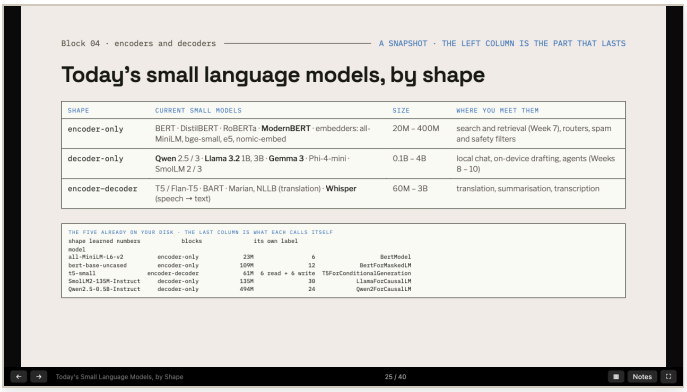
If the job is to write, chat, explain or follow instructions, the text does not exist yet. Reach for a **decoder**. It was trained to produce text one token at a time, and one instruction-tuned model handles many such tasks through the prompt.

If the job is to convert one text (or audio) into another, the input exists and the output does not. The classic answer is an **encoder–decoder**, which reads the whole input first and then writes. Increasingly the practical answer is a decoder with the input pasted into its prompt, which is what “summarise this” in a chat window is.

Two refinements for the real world. The shapes compose: Week 7’s retrieval uses an encoder to *find* and a decoder to *write*, and a good support system might use an encoder to route a ticket and a decoder to draft the reply. And “which shape” is only the first question; the second — how *big* — is Block 5’s, and it turns on cost, latency and privacy as much as on quality.

GO FURTHER

- Hugging Face, “Task summary” — the library’s own map from jobs to model types, with runnable examples.



SLIDE 25 OF 40 · BLOCK 04 · ENCODERS AND DECODERS

A SNAPSHOT · THE LEFT COLUMN IS THE PART THAT LASTS

Today's small language models, by shape

A snapshot of today's small language models, grouped by shape. The names will change within the year; the left column will not.

Small language models (SLMs) are models small enough to run on a laptop or one modest GPU — roughly, under a few billion learned numbers. This slide groups the current ones by shape, and the shape is the part of the table that lasts; treat the names and sizes as a snapshot taken when this deck was written.

Encoder-only, 20 to 400 million. BERT and its relatives (DistilBERT, RoBERTa) are the classics. ModernBERT (December 2024) is the same idea rebuilt with everything learned since — much faster, much longer inputs. The embedding models (all-MiniLM, bge-small, e5, nomic-embed, EmbeddingGemma) are encoders trained for one job, turning a text into one vector, and they run search, routing, filtering and de-duplication in production everywhere.

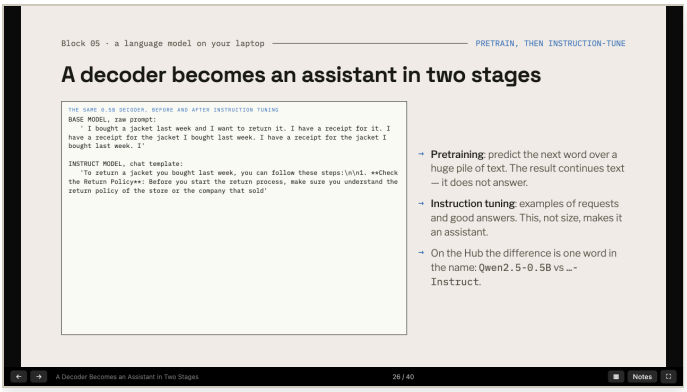
Decoder-only, 100 million to 4 billion. Qwen 2.5 and 3 (0.5B upward), Llama 3.2 (1B and 3B), Gemma 3 (270M to 4B), Phi-4-mini (3.8B), SmolLM 2 and 3 (135M to 3B). These are the local chat and drafting models, and the ones Weeks 8 to 10 will run as agents. The lecture notebook uses the Qwen and SmolLM ones.

Encoder–decoder, 60 million to 3 billion. T5 and Flan-T5, BART, the translation models (Marian, NLLB), and Whisper for speech to text.

The bottom table is your own disk. The notebook counts the five models already in the Hugging Face cache and prints what each one calls itself. The last column is the giveaway: `…ForMaskedLM` is an encoder trained by blank-filling (a bare `BertModel` is the same encoder without the blank-filling layer, which is all an embedder needs); `…ForCausalLM` is a decoder trained on the next word — *causal* means "may only look back"; `…ForConditionalGeneration` is an encoder–decoder. You will choose between exactly those class names in Assignment 5. Notice also the depth column: the 135M SmolLM2 is 30 blocks deep and the 494M Qwen is 24 — small models are not shallow, they are narrow.

GO FURTHER

- Model cards, one per shape, each with the exact command to load it: Qwen3-0.6B · Llama-3.2-1B-Instruct · gemma-3-1b-it · Phi-4-mini-instruct · ModernBERT-base · bge-small-en-v1.5 · flan-t5-base · whisper-small.
- Qwen Team, "Qwen2.5 Technical Report" (2024) — for the family used throughout the notebook; §2 describes the architecture in the vocabulary of this week.



SLIDE 26 OF 40 · BLOCK 05 · A LANGUAGE MODEL ON YOUR LAPTOP
PRETRAIN, THEN INSTRUCTION-TUNE

A decoder becomes an assistant in two stages

The same half-billion decoder, twice: the base model continues the customer's text and loops; the instruct model answers with steps. The difference is a second stage of training, not size.

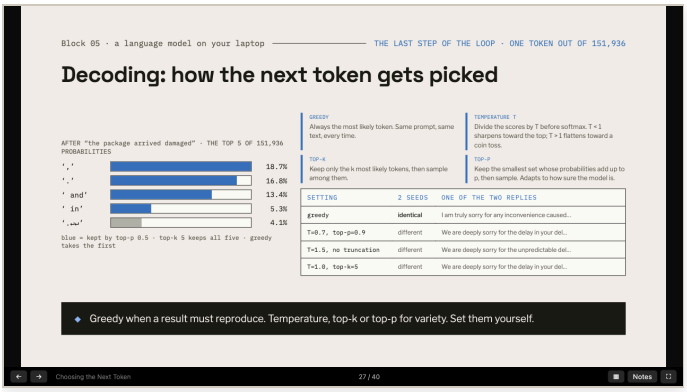
Block 5 moves from the architecture to the thing you actually download, and the first distinction is the one that trips up the most people. A decoder comes out of its main training run — **pretraining**, next-word prediction over a vast pile of text — as a **base model**. A base model does exactly what it was trained to do: **continue the text**. Give it a customer's message and it writes more of the customer's message. On the slide, the base Qwen2.5-0.5B, handed *I bought a jacket last week and I want to return it*, continues *I have a receipt for it. I have a receipt for the jacket I bought last week. I have a receipt for the jacket…* and falls into a loop. Nothing is broken; nothing in next-word prediction says that a question deserves an answer.

The **instruct** model is the same 494 million learned numbers after a second, much smaller stage of training — **instruction tuning** — on examples of requests paired with good answers (and, in most current models, a further stage in which human preferences between answers are used to adjust it). Handed the same message through its chat template, it answers: *To return a jacket you bought last week, you can follow these steps: 1. Check the Return Policy…* That behaviour is installed by the second stage. It is not a matter of size; the two models on the slide are identical in size.

Three practical consequences. On the Hub, the difference is a suffix in the name — `-Instruct`, `-it`, `-chat` — and a model without one will not follow instructions, no matter how the prompt is written; check the suffix before you debug your prompt. Second, instruction tuning is where the model's *manners* come from — its tendency to refuse, to hedge, to format answers as lists — so two instruct models with the same base can behave quite differently. Third, Week 6 builds on this directly: fine-tuning is a third stage of the same kind, with your own examples.

GO FURTHER

- Ouyang et al., InstructGPT (2022) — the paper that established the pretrain → instruction-tune → preference-tune recipe; Figure 2 is the whole method.
- Jurafsky & Martin, ch. 9 "Post-training" — optional reading on the course list; the textbook account of the same stages.



SLIDE 27 OF 40 · BLOCK 05 · A LANGUAGE MODEL ON YOUR LAPTOP

THE LAST STEP OF THE LOOP · ONE TOKEN OUT OF 151,936

Decoding: how the next token gets picked

At every step the model hands back a probability for each of its 151,936 tokens. Decoding is the rule for picking one: greedy, temperature, top-k, top-p. Two runs per setting show what each rule does to an apology.

Generation is Week 4's loop, and this slide is the step the loop glosses over: *choose*. The model emits a probability for every one of its 151,936 tokens, and something has to pick. The bars are real: after *the package arrived damaged*, Qwen2.5-0.5B-Instruct gives a comma about 19%, a full stop 17%, *and* 13%, *in* 5%, a paragraph break 4%, and spreads the remaining 42% across everything else. Four rules for picking from that list.

Greedy takes the most likely token every time. The same prompt always produces the same text, which is what you want when a result has to be reproducible; Assignment 5 pins every decoding parameter for that reason. Its weakness is repetition, the loop the base model fell into on the previous slide.

Temperature divides the model's scores by T before the softmax. Below 1 the distribution sharpens toward the top choice; above 1 it flattens toward a coin toss among everything; at 0 it is greedy. It does not change which tokens are candidates, only how lopsided the draw is.

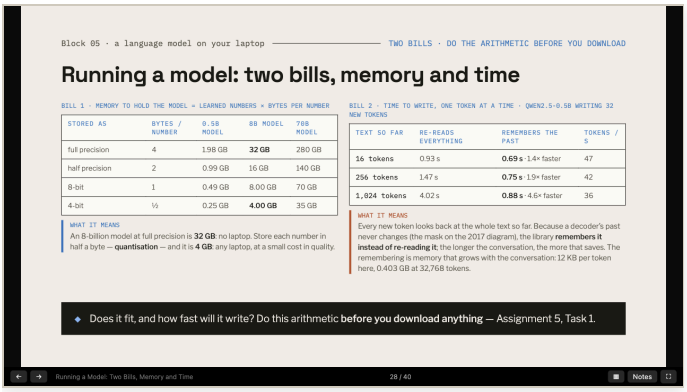
Top-k keeps only the k most likely tokens and samples among them. With $k = 5$ that is exactly the list on the slide. Its weakness is that k is fixed: five candidates is generous when the model is sure and stingy when it is not.

Top-p, also called nucleus sampling, keeps the smallest set of tokens whose probabilities add up to p and samples among those. Here $p = 0.5$ keeps four tokens, because the model is unsure; on a confident prediction the same p would keep one. It adapts to the model's confidence, which is why it is the common default.

The table on the right is the measurement, from the notebook: one apology prompt, four settings, two seeds each. Greedy produced the identical sentence twice. Temperature 0.7 with top-p 0.9 produced two different, sensible apologies. Temperature 1.5 with nothing cut off produced two different pieces of nonsense, which is what a flattened distribution sounds like. Top-k 5 at temperature 1.0 was sensible again, because the truncation did the work the temperature undid. When two runs of the same prompt disagree, the model is not broken; sampling is doing what it was asked to do. One warning, which the Hugging Face slide returns to: every checkpoint ships its own defaults for these knobs, and they apply whether or not you asked. Comparing two models, set the rule yourself.

GO FURTHER

- Hugging Face, “Generation strategies” — greedy, sampling, top-k, top-p and beam search, with the exact parameter names the notebook uses.
- Holtzman et al., “The Curious Case of Neural Text Degeneration” (2019) — the paper that introduced top-p and showed why greedy decoding degenerates into repetition.



SLIDE 28 OF 40 · BLOCK 05 · A LANGUAGE MODEL ON YOUR LAPTOP

TWO BILLS · DO THE ARITHMETIC BEFORE YOU DOWNLOAD

Running a model: two bills, memory and time

Two bills decide whether a model runs on your machine. Memory to hold it is learned numbers × bytes per number: an 8-billion model is 32 GB at full precision and 4 GB at four bits. Time to write is one token at a time, each looking back at the whole text; the library remembers the past instead of re-reading it, and that memory grows with the conversation.

Everything on this slide is arithmetic you can do before downloading anything, and Assignment 5's first task is doing it.

Bill 1 — memory to hold the model. Before it reads a single word, a model has to sit in memory, and the cost is one multiplication: learned numbers × bytes per number. The left-hand table does it for three sizes and four ways of storing a number. At **full precision** — four bytes per number, the format the notebook has mostly used — the half-billion model takes about 2 GB (the notebook confirms 1.976 GB actually resident), an 8-billion model 32 GB, a 70-billion model 280 GB. Read the 8B column against a laptop: 32 GB of weights alone, before any working memory, does not fit on most of them. At **half precision**, 16 GB. At **8-bit**, 8 GB. At **4-bit**, 4 GB — comfortable on almost anything. The 8-bit and 4-bit rows are **quantisation**: storing each learned number in fewer bits. It is the entire reason multi-billion-number models run on ordinary hardware, and it costs a little quality — more at 4-bit than at 8, and more on reasoning than on formatting. Week 6 meets it again when fine-tuning is done on a 4-bit base model (QLoRA).

Bill 2 — time to write. Generating text is the loop you wrote in Week 4: run the model, take the probabilities for the next token, choose one (the previous slide), append it, repeat. Done naively, each pass re-reads the **entire** text so far to produce one new token — and the notebook's causal-mask test (§3.2) shows that almost all of that work would produce exactly the same vectors as the pass before, because a decoder's past cannot change. So the library keeps them: at every block it stores what attention computed for every earlier token (the *KV cache*, for keys and values) and lets each new token compute only its own. Same output, a fraction of the work. The right-hand table measures it on the half-billion model writing 32 new tokens: after a 16-token prompt the saving is small (1.4×); after 256 tokens, 1.9×; after 1,024 tokens, **4.6×**. The longer the conversation, the more re-reading is avoided. Extrapolate to the 100,000-token contexts of current models and recomputing the past every step is not a slow option but an impossible one. Every hosted model you call in Week 6 does this — it is why providers price a *cached* prompt prefix more cheaply than a fresh one.

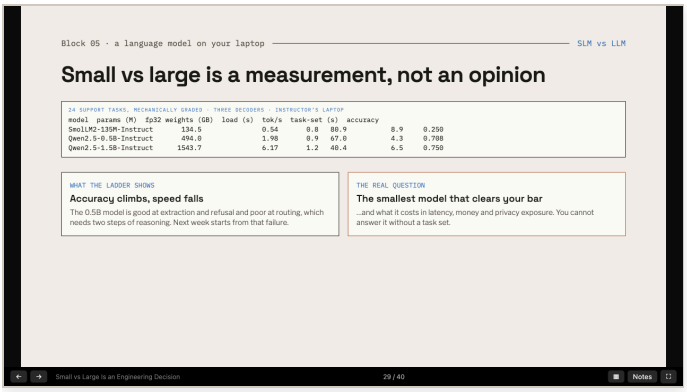
The remembering is not free. It trades arithmetic for memory, and that memory grows with every token of the conversation: about 12 KB per token for this model, so 0.4 GB for a 32,768-token conversation, and 3.2 GB if eight users are served at once. The notebook shows the design decision that keeps the

number that small — this model's 14 attention heads share only 2 sets of stored keys and values (*grouped-query attention*), a 7× saving baked into almost every model released since 2023 purely to make serving cheaper. Put this beside the appendix's RNN-vs-transformer table and you have the two costs of long context: attention's work grows with the square of the length, and the cache's memory grows with the length. Context is the expensive resource in this technology, and Week 7 is about spending it deliberately.

What the slide leaves out, deliberately. Speed also depends on bytes: generating one token means reading every learned number once, so tokens per second is bounded by memory bandwidth as much as by arithmetic, and half-precision weights are usually faster as well as smaller. Assignment 5, Task 2, is measuring both. And the practical side of the Hugging Face library — `pipeline`, the `Auto*` classes, the download cache and its disk cost, and above all the **chat template** an instruct model expects — is not in the lecture any more; notebook §5 covers it, and you will need it for the assignment. The one rule to carry from there: an instruct model was tuned on one exact string format, so always let `apply_chat_template` write your prompt rather than sending a bare sentence.

GO FURTHER

- Hugging Face, "Quantization" overview — the practical options for the 8-bit and 4-bit rows.
- Hugging Face, "KV cache strategies" — the library's account of Bill 2, including the cache types the assignment may meet.
- Hugging Face, "Chat templates" — the rule in the last paragraph, with the literal strings.
- kipply, "Transformer Inference Arithmetic" (2022) — optional on the course list; the back-of-envelope for everything on this slide.



SLIDE 29 OF 40 · BLOCK 05 · A LANGUAGE MODEL ON YOUR LAPTOP

SLM vs LLM

Small vs large is a measurement, not an opinion

“Use the biggest model” is not engineering. The question is the smallest model that clears your quality bar, and what it costs in speed, memory, money and privacy — and you cannot answer it without a task set.

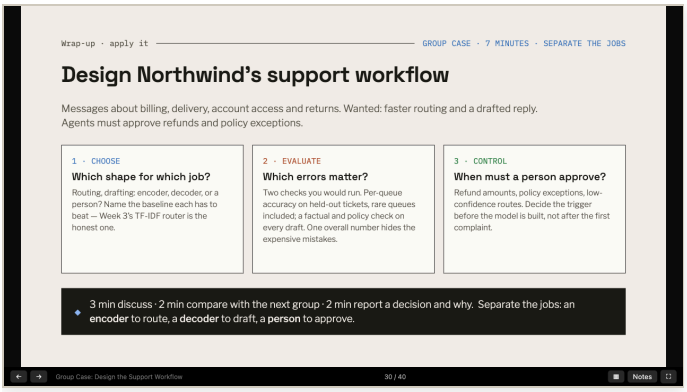
The last slide of Block 5 turns everything before it into a decision. Three decoders of increasing size are run on the same 24 Northwind support tasks in five categories — extract a field, produce a required format, refuse an out-of-scope request, route a ticket, judge sentiment — each graded by a mechanical rule so the score is reproducible. Grading by rule is a deliberate constraint: it makes the number honest and its blind spots discussable, which is where Week 10’s evaluation material starts.

Read the table left to right. **Accuracy climbs with size**: 0.25 for the 135-million SmolLM2, 0.71 for the half-billion Qwen, 0.75 for the 1.5-billion Qwen. **Speed falls**: about 81, 67 and 40 tokens per second on the instructor’s laptop. **Memory rises**: 0.5, 2.0 and 6.2 GB of weights at full precision. The per-category breakdown in the notebook is the more useful reading: the half-billion model is perfect on extraction and refusal, adequate on sentiment, weak on format compliance, and poor on routing (2 of 6) — and the model three times its size fixes sentiment but **not** routing. Routing here needs two steps of reasoning (identify the issue, then map it to a queue), and the model collapses onto one label. That failure is not a size problem; it is where Week 6 starts, with prompting and structured output.

The engineering lesson, on the right: find the **smallest model that clears your quality bar**, and know what it costs in speed, memory, money and privacy exposure. A small model run locally costs nothing per call, keeps the data on your machine, and answers in milliseconds; a frontier model behind an API is far more capable and charges by the token. Neither is the right answer in general. The right answer depends on your tasks, which is why the first thing to build is a task set — and why Assignment 5 has you build this comparison on your own hardware, with a frontier model alongside, and write the recommendation.

GO FURTHER

- Assignment 5 README — the task set, the models, and the frontier comparison you will run.
- Hoffmann et al., “Training Compute-Optimal Large Language Models” (Chinchilla, 2022) — why a smaller model trained on more data can beat a larger one, which is the reason today’s small models are as good as they are.



SLIDE 30 OF 40 · WRAP-UP

GROUP CASE · 7 MINUTES · SEPARATE THE JOBS

Design Northwind's support workflow

Seven minutes in groups: choose a shape for routing and for drafting, name the baseline each must beat, pick two checks that would show it works, and decide when a person must approve.

The week ends where it began, with Northwind's support queue. Messages arrive about billing, delivery, account access and returns. Management wants two things: faster routing, and a drafted reply the agent can edit. Agents must approve every refund and every policy exception. Groups of two or three have seven minutes: three to discuss, two to compare with the next group, two to report a decision and its justification.

Choose. Which shape for which job, and against what baseline? Routing is a sort job over text that already exists: Block 4's rule of thumb says encoder, and the honest baseline it must beat is Week 3's TF-IDF router, which scored 0.89 on the same tickets in Week 4's notebook. Drafting is text that does not exist yet: a decoder, handed the ticket and the approved policy text in its prompt. A prompted decoder is a legitimate routing candidate too; the point is to name the comparison, not to guess the winner.

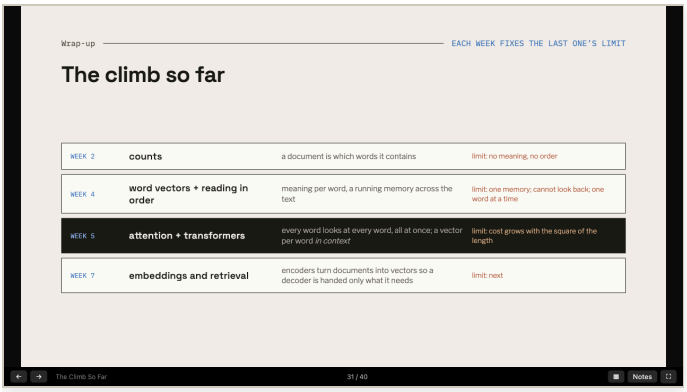
Evaluate. Which errors matter, and what two checks would you run? For routing, per-queue accuracy on held-out tickets, with the rare queues reported separately: one overall number hides a fraud queue that is never routed correctly, and slide 29 showed a model that looked reasonable overall and collapsed on routing. For drafting, a factual check (does the reply mention only what the ticket and the policy support?) and a policy check (does it promise a refund or an exception?), read by a person on a weekly sample.

Control. When must a person approve? Refund amounts above a threshold, any policy exception, and any route the model is unsure about. Decide the trigger before the model is built. A fluent reply that offers a replacement to the customer who has just refused one is a business failure, however good the model's score.

One defensible design, for the debrief. Routing: an encoder classifier, compared with the TF-IDF baseline on per-queue results, cost and latency, and adopted only if it wins on the queues that matter. Drafting: a decoder with approved source information in the prompt, agent approval on every refund, and a weekly sample read for factual and policy errors. Dates and order numbers: a deterministic parser with explicit validation, because rules can be tested and a model cannot promise a format. Different jobs, different tools, and a person in the loop wherever money moves.

GO FURTHER

- Assignment 5 README — Task 5 is this case with numbers: the frontier invoice against the fully-loaded local machine.
- Jain & Wallace, "Attention is not Explanation" (2019) — why an attention pattern is not the evidence a reviewer needs.



SLIDE 31 OF 40 · WRAP-UP

EACH WEEK FIXES THE LAST ONE'S LIMIT

The climb so far

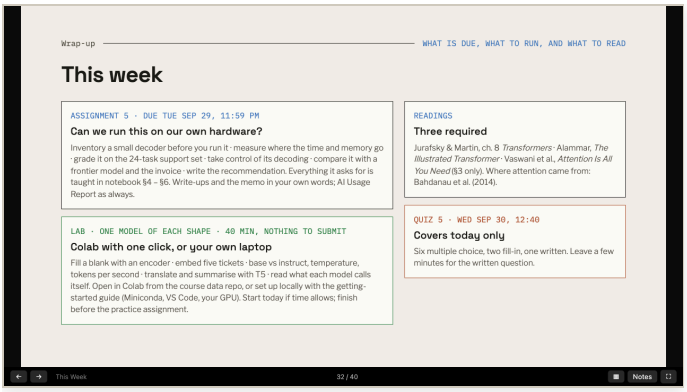
Each week fixes the last one's limit: counts, then word vectors and reading in order, then attention and transformers, then — next — retrieval so that a decoder is handed only what it needs.

The wrap-up puts the week on the course's ladder. **Week 2, counts**: a document is which words it contains — no meaning, no order. **Week 4, word vectors and reading in order**: meaning per word and a running memory across the text — but one memory, no way to look back, one word at a time. **Week 5, attention and transformers**: every word looks at every word, all at once, and each word gets a vector *in context*. The limit this week's solution carries is on the last row of slide 35: cost grows with the square of the length, and the cache grows with the length. Context is expensive.

Week 7 is how that bill is avoided rather than paid: encoders (notebook §3.1) turn documents into vectors, a search finds the few documents relevant to a question, and a decoder is handed only those. The two shapes from Block 4 working together is the whole architecture of a retrieval system, and it is why this week's encoder/decoder distinction was worth thirty minutes.

Between here and there, **Week 6** takes the failures from slide 29 — routing collapsing onto one label, format compliance at 60% — and fixes what can be fixed by prompting, few-shot examples and structured output, then shows fine-tuning for the cases that cannot. The decoders on slide 25 are the models Week 6 will prompt.

If you want a single sentence to carry out of the week: *a transformer is a stack of blocks in which every word asks a question of every other word and adds what it learns to itself; an encoder lets the questions look both ways and returns vectors, a decoder lets them look only back and writes.* Everything else on these forty slides is either a measurement of that sentence or a consequence of it.



SLIDE 32 OF 40 · WRAP-UP

WHAT IS DUE, WHAT TO RUN, AND WHAT TO READ

This week

Assignment 5 asks whether a small model can run on your own hardware, measured; a forty-minute lab puts one model of each shape in your hands; three required readings; Quiz 5 next Wednesday covers today only.

Assignment 5 is due Tuesday 29 September at 11:59 PM. It applies Block 5 end to end on your own machine: inventory a small decoder before you run it (slide 28's arithmetic, Task 1); measure where the time and memory go (Task 2); grade it on the 24-task support set (Task 3); control its decoding rather than accepting the checkpoint's defaults (Task 4); compare it with a frontier model and the invoice (Task 5); write the recommendation (Task 6). Everything it asks for is taught in notebook sections 4 to 6. As on every assignment: the write-ups and the memo in your own words, and the AI Usage Report with the complete transcript.

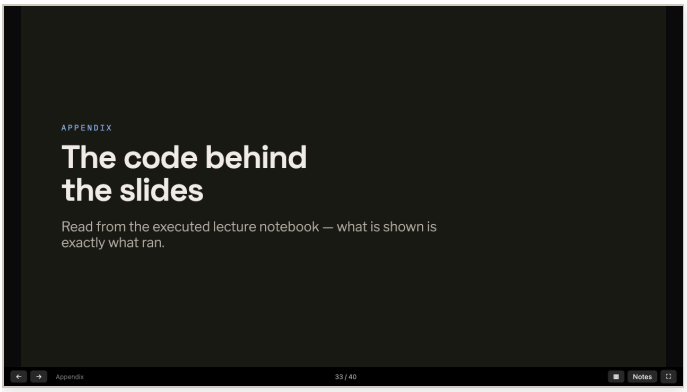
Lab. *One model of each shape*, forty minutes, follows slide 25: fill a blank with BERT and prove it read the words after the blank; embed five tickets with MiniLM; watch the base Qwen continue a message while the instruct Qwen answers it, then sweep the temperature and time tokens per second; translate and summarise with T5, including a language it never saw; print what each model calls itself. It opens in Google Colab with one click from the course data repository, or runs locally after the getting-started guide (Miniconda, VS Code, your GPU). Nothing to submit; bring the results sheet.

Readings. Three required: Jurafsky and Martin's transformer chapter; Alammar's *Illustrated Transformer*; and §3 of *Attention Is All You Need*, which you will find readable after this lecture and not before. Bahdanau (2014) is listed as historical, where attention came from, and is the paper Block 2 rebuilt. An optional video by Grant Sanderson covers Block 3 in an hour and is the source of the question/answer/what-to-add way of telling it.

Quiz 5 is next Wednesday at 12:40, in class, covering today only: six multiple choice, two fill-in, one written question worth two of the ten points. The written question will ask you either to explain a mechanism from Blocks 2–3 or to choose a model shape for a described job and defend it in Block 4's terms. On the previous quizzes many written answers stopped mid-sentence; leave a few minutes for it, and answer each part in one clear sentence.

GO FURTHER

- Week 5 lab, open in Colab — the notebook; the getting-started guide for running it on your own machine.
- Week 5 readings — the full annotated list, with what to read in each item.
- Week 5 practice assignment — eight short steps on a 135-million-number model; do it before the graded assignment.

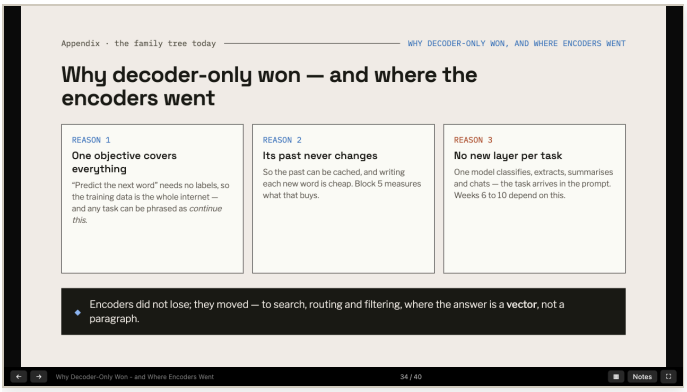


SLIDE 33 OF 40 · APPENDIX

The code behind the slides

The code behind the three models on the slides, read straight from the executed lecture notebook.

The appendix holds the code for the two models that carry Blocks 2 and 3. It is read from the executed lecture notebook at the moment the deck is built, so what is shown is exactly what ran. You are not expected to reproduce any of it from memory; you are expected to be able to open the notebook, find these cells, and recognise each line as one of the ideas on the slides.



SLIDE 34 OF 40 · APPENDIX

WHY DECODER-ONLY WON, AND WHERE ENCODERS WENT

Why decoder-only won — and where the encoders went

Decoder-only did not win because it is the best shape at any one task. It won on three practical grounds — and the encoders did not lose; they moved to the jobs where the answer is a vector.

Every frontier chat model you have used — GPT, Claude, Gemini, Llama — is decoder-only. It is worth being clear about why, because the reason is not that decoders are better at everything. On a classification benchmark a same-size encoder usually still wins; on translation an encoder–decoder is still excellent. Decoder-only won on three practical grounds.

One objective covers everything. "Predict the next word" needs no labels, so the training data can be essentially all the text there is, and every other task — classify, extract, summarise, translate, chat — can be phrased as *continue this text*. An encoder needs a separate small classifier bolted on for each task; a decoder needs a prompt.

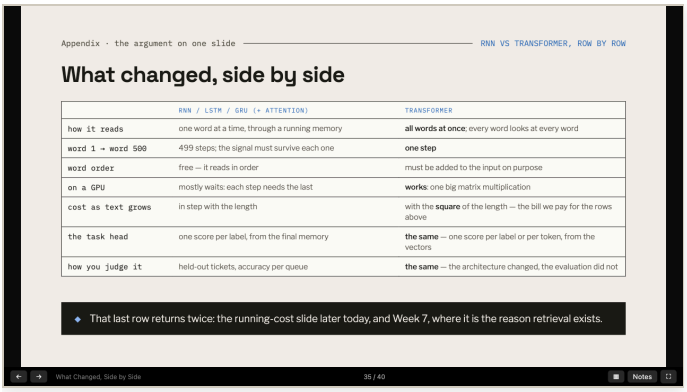
Its past never changes. Slide 28 proved it: a decoder's vectors for earlier words are untouched by later ones. So they can be cached, and writing the next word costs only that word's work. Slide 34 measures what that buys. An encoder cannot generate at all without recomputing everything at every step.

No new layer per task. One trained model serves classification, extraction, summarisation and conversation, with the task arriving in the prompt. Weeks 6 to 10 of this course depend on exactly this flexibility: prompting, tool use, agents and multi-agent systems are all the same decoder being asked different things.

The bottom band is the half people forget. Encoders did not disappear. They moved to the jobs where the answer is a **vector**, not a paragraph: search and retrieval, routing and classification at scale, spam and safety filters, de-duplication. Week 7's retrieval systems use an encoder to find the documents and a decoder to write from them — the two shapes working together, each doing the job it is built for.

GO FURTHER

- Kaplan et al., "Scaling Laws for Neural Language Models" (2020) — the finding that decoder-only models kept improving predictably with size and data, which is the economic reason behind reason one.
- Ouyang et al., "Training language models to follow instructions with human feedback" (InstructGPT, 2022) — how a text-continuing decoder was turned into an assistant; slide 26 returns to this.



SLIDE 35 OF 40 · APPENDIX

RNN VS TRANSFORMER, ROW BY ROW

What changed, side by side

Five rows that hold the whole argument: how it reads, how far apart two words are, where order comes from, what a GPU can do, and what it costs as text grows.

This table is the block's summary and the one slide to revisit before the quiz. Each row is one of the dominoes.

How it reads. The RNN reads one word at a time through a running memory; the transformer processes all words at once, every word looking at every other. This is the fix for limit 3.

Word 1 to word 500. In an RNN the information has to survive 499 steps of the memory being rewritten — and the training signal that would teach the model to preserve it travels back along the same 499 steps, shrinking as it goes. In a transformer the two words are one attention step apart. This is why transformers handle long-range relationships (a pronoun and its referent three paragraphs apart) that recurrent models struggled with even with gates.

Word order. Free for the RNN; must be added on purpose for the transformer (slide 20). A real cost, but a small and solved one.

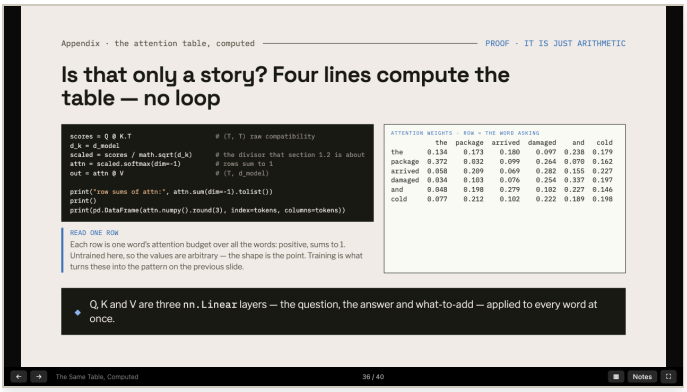
On a GPU. The RNN mostly waits, because each step needs the last; the transformer's attention is one large matrix multiplication and keeps the hardware busy. This is the economic fact behind the last eight years: the models could be scaled because the hardware could be used.

Cost as text grows. In step with the length for the RNN; with the **square** of the length for the transformer, because the attention table has an entry for every pair of words. This row is the bill for all the rows above it, and it returns twice: on slide 28, where the cache exists to avoid paying it repeatedly while writing, and in Week 7, where retrieval exists so that a model is handed only the few documents it needs rather than all of them.

If you take one thing from Block 3, take this table.

GO FURTHER

- Vaswani et al. (2017), Table 1 — the paper's version of this comparison, with the complexity formulas.



SLIDE 36 OF 40 · APPENDIX

PROOF · IT IS JUST ARITHMETIC

Is that only a story? Four lines compute the table — no loop

A fair worry — is that only a story? — answered with four lines of PyTorch that compute the table on real vectors, no loop anywhere.

After a slide of drawn dots and named roles, it is reasonable to ask whether any of this is real arithmetic. The code answers. Six words, each a 64-number vector. Three learned layers make Q, K and V. Then:

1. `scores = Q @ K.T` — every query against every key, one matrix multiplication, giving a 6×6 table of raw scores;
2. divide by the square root of the vector size (slide 36 says why, briefly);
3. `softmax` across each row — the row becomes a budget that sums to 1;
4. `@ V` — each row's budget blends the values into one new vector per word.

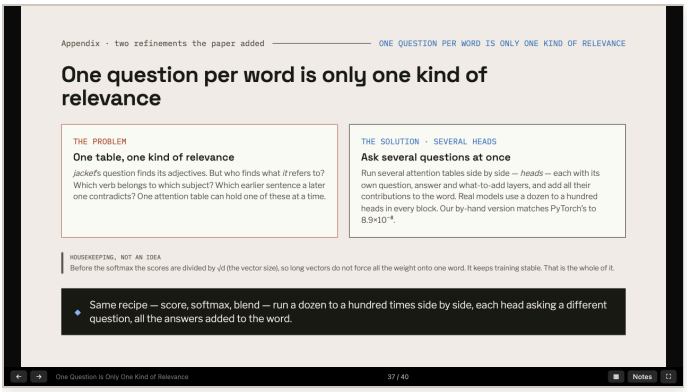
The printed table is the attention pattern that comes out. The row sums print as 1 to within floating-point noise, which is the softmax doing its job. The numbers themselves mean nothing yet — this is an **untrained** layer with random weights, so *package* attending 0.372 to *the* is an accident. Training is what turns this arbitrary table into the useful one drawn on the previous slide, by adjusting the three layers until the right words match.

Two things to notice. There is **no loop** over the words anywhere in those four lines; all six words are handled together, which is exactly the property Block 2 said the RNN lacked. And the three layers are ordinary `nn.Linear` layers — the same kind of layer you counted in Week 3's feed-forward network — so a transformer is built from parts you already know, arranged so that words can look at each other.

The notebook also checks the arithmetic against PyTorch's own fused implementation (`F.scaled_dot_product_attention`) and gets the same answer. That is worth knowing for Assignment 5: the library does this in one call, but it is doing nothing you could not write in four lines.

GO FURTHER

- PyTorch `nn.MultiheadAttention` and `F.scaled_dot_product_attention` — the packaged versions of these four lines.
- Andrej Karpathy, "Let's build GPT: from scratch, in code, spelled out" (2023) — two hours that build exactly this, then a whole decoder, in plain PyTorch.



SLIDE 37 OF 40 · APPENDIX

ONE QUESTION PER WORD IS ONLY ONE KIND OF RELEVANCE

One question per word is only one kind of relevance

One attention table holds one kind of relevance at a time. So run several side by side — heads — each asking its own question, and add all their contributions to the word.

Take the story seriously and a problem appears at once. *Jacket*'s question finds its adjectives. But a sentence carries many kinds of relevance at the same time: which noun a pronoun refers to, which subject belongs to which verb, which earlier clause a later one contradicts, which date a "that day" points at. One attention table — one set of question, answer and what-to-add layers — can express one of these at a time, because it produces one budget per word.

The solution is almost embarrassingly direct: run **several** attention tables side by side. Each is called a **head**, has its own three learned layers, produces its own pattern, and contributes its own weighted values, and all of the contributions are added to the word's vector. Real models use a dozen to a hundred heads in every block; the small model on slide 38 has 14 per block, 24 blocks deep. In the code, the heads are implemented by slicing each vector into pieces and running attention on each piece, so the total cost is the same as one big head — a re-slicing of a fixed budget, not an addition to it. The notebook builds this by hand and matches PyTorch's `nn.MultiheadAttention` to eight decimal places.

The footnote is the second refinement, demoted on purpose. Before the softmax, the raw scores are divided by the square root of the vector size (√d). With long vectors, dot products can grow large, and a softmax of large scores puts almost all of its weight on one word, which makes learning stall. Dividing keeps the weights spread while the model is learning. It is housekeeping — an engineering detail that makes training stable — and the lecture gives it exactly one sentence because it is not an idea about language. It is mentioned at all because you will see the √d in every diagram of attention you ever meet, and should know it is nothing to be afraid of.

GO FURTHER

- Vaswani et al. (2017), §3.2.2 "Multi-Head Attention" — three paragraphs, and the footnote on why the √d.
- Clark et al., "What Does BERT Look At?" (2019) — evidence that individual heads in a trained model do specialise (some track syntax), and that many do not.



SLIDE 38 OF 40 · APPENDIX

REAL ATTENTION WEIGHTS · QWEN2.5-0.5B, 24 BLOCKS

Real attention, in a real model

The same idea at scale: real attention weights from a half-billion-number model on one sentence — local patterns in early blocks, a strange habit in deep ones, and nothing above the diagonal.

Everything in Blocks 2 and 3 ran on models that train in seconds. This appendix slide opens the window from slide 15 on a real language model: Qwen2.5-0.5B-Instruct, 494 million learned numbers, 24 blocks of 14 heads each, reading *The jacket that the customer returned last week was damaged.* Each panel is one block's attention pattern, averaged over its 14 heads; rows are the word asking, columns the words being looked at, darker means more of the budget.

Three things to see, and the notebook prints the numbers behind them.

Early blocks look nearby. In the first block the strongest links are between neighbouring words and small, local phrases; the pattern hugs the diagonal. Layer 2, head 9, for example, sends 99% of *week*'s budget to *last* — the phrase *last week* being assembled.

Deep blocks develop a strange habit. Look at the left-hand column of the deeper panels: a great deal of every row's budget goes to the *first* token. The notebook measures it — the average attention paid to token 0 climbs from 0.21 in the first block to 0.89 in block 11, and averages 0.6 across all 24 blocks. This is a known phenomenon called the **attention sink**: because every row must sum to 1, a head that has nothing useful to look up needs somewhere to park its budget, and models learn to dump it on the first position. It is a good example of why attention weights are a window, not a complete explanation — a large weight can mean "important" or "nowhere better to look".

Nothing above the diagonal. In every panel the upper-right triangle is empty. This model never lets a word look at the words *after* it. That is not a coincidence of training; it is enforced, and Block 4 explains why — it is the property that makes this a *decoder*, a model that can write.

GO FURTHER

- Xiao et al., "Efficient Streaming Language Models with Attention Sinks" (2023) — where the attention-sink effect was named and explained; §3.1 is readable on its own.
- Alammar, "The Illustrated GPT-2" (2019) — the decoder's attention drawn one token at a time.



SLIDE 39 OF 40 · APPENDIX

NOTEBOOK §0.3 · THE ATTENTION SWITCH

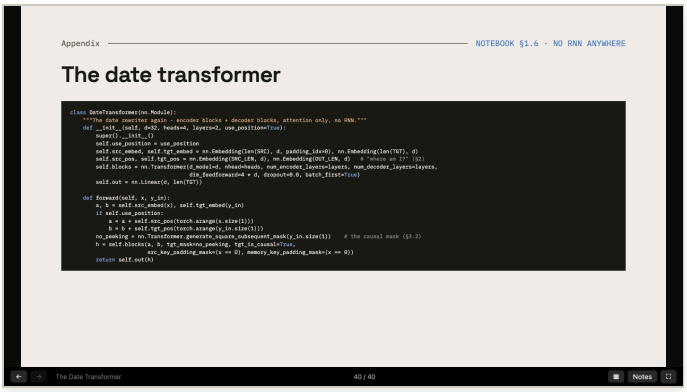
Attention in the date model: three lines

Three lines do the work of attention in the date model: score every input position against the decoder's state, softmax into weights, blend the encoder's memories. With the switch off, the context is the same summary at every step.

This is the decoder loop of the `DateRewriter` from Block 2 — the model that scored 3.6% without attention and 97.6% with it. The three lines under `if self.use_attention:` are slide 12's three steps in PyTorch.

The first line **scores**: the decoder's current state (`state`, the question) and every encoder memory (`enc_out`) are each passed through a small learned layer, added, squashed with `tanh`, and reduced to one number per input position. This is the *additive* scoring Bahdanau used; a raw dot product, as on slide 37, would also work. The second line **softmaxes** the scores into weights that sum to 1, after setting the scores of padding positions to a huge negative number so that empty slots get a weight of zero — the same masking trick that the causal mask uses on slide 22. The third line **blends**: each encoder memory multiplied by its weight, summed, giving one *context* vector for this step.

The `else` branch is the whole of the no-attention model: `context = summary`, the encoder's final memory, identical at every step. Everything below the `if` is shared — the decoder cell takes the previous character and the context, updates its state, and scores the next character. That is why the two models have identical learned-number counts: the attention layers exist in both; one of them never calls them.



SLIDE 40 OF 40 · APPENDIX

NOTEBOOK §1.6 · NO RNN ANYWHERE

The date transformer

The date transformer: embeddings for the characters, embeddings for the positions, PyTorch's packaged encoder and decoder blocks, and one output layer. There is no RNN anywhere in it.

This is the date transformer of Block 3, whose accuracy is on slide 20, and every line maps to a slide. The two `nn.Embedding` tables turn input and output characters into 32-number vectors — Week 4's lookup table. The two *position* embeddings (`src_pos`, `tgt_pos`) are slide 20: one learned vector per position, added to the character's vector, and the flag `use_position` is the switch that produced the 97.8% versus 34.5% comparison. `nn.Transformer` is slides 16 to 22 packaged: two encoder blocks and two decoder blocks, four heads each, a feed-forward width of four times the vector size, and the skip connections and normalisation built in. The output layer turns the decoder's final vectors into one score per possible character.

Two lines carry Block 4 in miniature. `generate_square_subsequent_mask` builds the causal mask — the masked attention box on slide 22 — so that each output position may look only at the characters written before it; it is passed as `tgt_mask`. And the encoder's output is handed to the decoder as its *memory*, which is what the decoder blocks attend across to: Block 2's idea, with the recurrence gone. The `src_key_padding_mask` and `memory_key_padding_mask` arguments hide the padding slots from attention, exactly as the `masked_fill` did in the previous appendix slide.

There is no recurrent layer anywhere in the class. Reading the input and writing the output are both done by attention, all positions at once.

Glossary

- **Attention** — score every item against a question, softmax the scores into weights that sum to 1, and blend the items by those weights. A summary built for one question.
- **Attention pattern** — the table of weights: rows are the words asking, columns the words answering, each row summing to 1.
- **Attention sink** — the habit of trained decoders of parking spare attention budget on the first token; a reminder that a large weight means "looked at", not always "important".
- **Base model / instruct model** — a decoder after pretraining alone (it continues text) versus the same model after instruction tuning (it answers).
- **Block** — one repeatable unit of a transformer: multi-head self-attention, then a feed-forward network applied to each word alone, each wrapped in a skip connection and normalisation. Blocks stack; depth is the count of them.
- **Bottleneck** — the single fixed-size memory through which an encoder–decoder without attention passes everything about its input.
- **Causal mask** — the rule that a position may look only at earlier positions. It makes a model a writer, makes every position a training example, and makes the past cacheable.
- **Context window** — the maximum number of tokens a model can attend over at once. Expensive: attention's work grows with its square and the cache with its length.
- **Decoder-only** — the shape in which every word may look only back; trained to predict the next word. GPT, Llama, Qwen, Gemma, and every frontier chat model.
- **Embedding** — a vector standing for a word (Week 4's lookup table) or, after an encoder, for a whole text (one vector per document, the engine of retrieval).
- **Encoder-only** — the shape in which every word may look both ways; trained by restoring hidden words. BERT, ModernBERT, the embedding models. Returns vectors, not prose.
- **Encoder–decoder** — an encoder reads the input both ways; a decoder writes the output looking back and across. T5, BART, Whisper, translation models.
- **Feed-forward network (FFN / MLP)** — the two-layer network inside each block, applied to each word on its own; holds about two thirds of a block's learned numbers, and most of what the model knows.
- **Head** — one attention table with its own question, answer and what-to-add layers. A block runs many side by side and adds all their contributions.
- **Instruction tuning** — the second stage of training, on request–answer pairs, that turns a text continuer into an assistant.
- **KV cache** — the stored keys and values of earlier tokens, kept so that writing each new token costs only that token's work. Trades arithmetic for memory.
- **Learned numbers / parameters** — the numbers training adjusts. "0.5B" = half a billion of them.
- **Position encoding** — the vector added to each word's vector to say where it sits; without it self-attention is order-blind.
- **Quantization** — storing each learned number in fewer bits (8 or 4 instead of 16 or 32) to fit a model into less memory, at a small cost in quality.

- **Query, key, value** — the three vectors each word makes from its own vector in self-attention: the question it asks, the answer it offers, and what it contributes if picked.
- **Self-attention** — attention in which the questions, answers and values all come from the same sequence: every word asks every other word.
- **Small language model (SLM)** — a model small enough to run on a laptop or one modest GPU; roughly, under a few billion learned numbers.
- **Softmax** — exponentiate a list of scores and divide by the total: the result is positive and sums to 1. Turns scores into a budget.
- **Token** — the unit a model reads: a word, a piece of a word, a character.
- **Transformer** — a stack of blocks over word-plus-position vectors, in which every word attends to every other; no recurrence anywhere.

The three experiments, restated

1. **The bottleneck and its fix (Block 2).** Same task, same data, same training, same 12,317 learned numbers: 3.6% without attention, 97.6% with it. A 30×-larger memory reaches 99.1% and stops working when inputs get longer.
2. **Attention without the RNN (Block 3).** A two-block transformer, 63,053 learned numbers, scores 97.8% — and 34.5% with position switched off.
3. **Who may look at whom (Block 4).** Change a sentence's last word: a decoder's earlier vectors change by exactly 0; an encoder's all change. BERT fills a blank differently when only the words after it differ.